Why a Parallel Program Equals Its Sequential Reading

The paralever proofs, written out in full

How to read this

This document proves one theorem: a parallel program whose branches do not interfere produces exactly the same answer as the sequential program that runs those branches one after another — under *every* possible interleaving, not just the ones we thought to try.

It assumes nothing. Every symbol is defined before it is used, every proof is given with its cases named and none of them waved away, and each difficult step is preceded by the same step worked by hand on concrete numbers. Where a hypothesis is doing real work, we also show what breaks when it is dropped.

It follows the sequence of the paper, section for section:

Here	Paper	Contents
§1	—	the question, and a pair of programs that answers it badly
§2	§3	programs as data; why they are trees; why induction is awkward
§3	§4	what a program touches; independence; the machine
§4	§5	the surface language and its decidable check
§5	§6	the theorem, and the equivalence read off it
§6	§7	cost: same work, fewer rounds
§7	—	the whole argument in nine lines, and where it breaks

A reader who wants the theorem as fast as possible can read §1–§3, skip §4 entirely, and go straight to §5. Nothing in §5 depends on §4; the dependency runs the other way.

Contents

1 The question **2**

2 Programs as data **3**
- 2.1 Memory . . . 3
- 2.2 Programs . . . 4
- 2.3 Induction over a nested inductive . . . 6
- 2.4 Why not Lean's `Task` . . . 7

3 What a program touches, and the machine that runs it **8**
- 3.1 The reference run . . . 8
- 3.2 Footprints . . . 9
- 3.3 The staggered measurement . . . 9
- 3.4 Independence . . . 10
- 3.5 Well-formedness: the hypothesis . . . 11
- 3.6 The real machine . . . 11
- 3.7 What we want to prove . . . 13
- 3.8 Locality: four lemmas . . . 14
  - 3.8.1 Frame . . . 14
  - 3.8.2 Locality . . . 14

3.8.3 Isolation . . . 16
3.8.4 No race, and locality of the obligation . . . 17

4 The surface language 17
4.1 Memory is declared, not allocated . . . 18
4.2 The footprint is computed, not proved . . . 19
4.3 The obligation is decided . . . 20
4.4 Pipelines and kernels . . . 20
4.5 Computed addresses and branches . . . 21
4.6 Proposing a parallelisation . . . 22

5 Canonical outcomes, and the equivalence 23
5.1 The theorem, and how to attack it . . . 24
5.2 Piece 1: a run out of a fork is in one of three states . . . 25
5.3 Piece 2: children's runs lift to the fork . . . 25
5.4 Piece 3: one step preserves the canonical outcome . . . 26
5.4.1 Why the two conclusions are proved together . . . 28
5.5 Piece 4: chaining . . . 28
5.6 Safety: the forward direction . . . 29
5.7 Liveness: the backward direction . . . 30
5.8 Parallel equals sequential . . . 31
5.9 Strengthenings . . . 32

6 Cost: the same operations, fewer rounds 32
6.1 Work . . . 33
6.2 Rounds: a second, derived relation . . . 33
6.3 A round is a legal sequence of ordinary steps . . . 35
6.4 max as a theorem . . . 36

7 The whole thing on one page 38
7.1 Nine lines . . . 38
7.2 What rests on what . . . 39
7.3 Where it breaks . . . 39

1 The question

Here are two programs. The first says "do these two jobs at the same time". The second says "do the first job, then the second".

$$\texttt{A[5]} := 1 \quad \| \quad \texttt{A[8]} := \texttt{A[7]} \qquad \text{against} \qquad \texttt{A[5]} := 1;\ \texttt{A[8]} := \texttt{A[7]}$$

Do they leave the same memory behind? Your instinct says yes, and here your instinct is right. But consider this pair, which differs in one character:

$$\texttt{A[5]} := 1 \quad \| \quad \texttt{A[8]} := \texttt{A[5]} \qquad \text{against} \qquad \texttt{A[5]} := 1;\ \texttt{A[8]} := \texttt{A[5]}$$

Run the sequential version from an all-zeros memory. Cell 5 becomes 1; then cell 8 copies cell 5, so cell 8 ends up holding 1. Now run the parallel version and let the right-hand job go first: it copies cell 5 while cell 5 is still 0, so cell 8 ends up holding 0.

So the answer to the question is "sometimes". Our job is to find the exact condition under which the answer is *always yes*, and then prove it — not test it, prove it, for every program, every starting memory, and every way a machine might interleave the work. Everything that

follows is the machinery needed to state that claim precisely enough to be provable, and then the proof.

Two remarks on what we are *not* doing, because both shape the development.

We are not proving the parallel program has no races and then hoping determinism follows. It does not: §7.3 exhibits a program whose every individual step is race-free at the instant it happens, and which still has two terminating runs that disagree. The hypothesis has to be stronger than race freedom, and §3.5 is where it gets defined.

We are also not proving a one-sided statement of the form "if the parallel program terminates then its answer is correct". That is what a soundness theorem usually gives, and it is satisfied by a program with no terminating runs at all. What we prove is an equality of outcome *sets*, which rules that out; the extra direction costs us §5.7.

2 Programs as data

A theorem relating a parallel program to a sequential one needs two things. Both programs must be objects the logic can quantify over — so we can say "for every program $p\ldots$" — and parallel composition must *mean* something the logic does not already know. Those two requirements pin the representation down almost completely, and this section says how.

2.1 Memory

We need to say what memory is. The simplest thing that works:

Definition 1 (Addresses, values, memory)**.** An address is a natural number, $\mathsf{Loc} := \mathbb{N}$. A value is drawn from a small universe Val of storable things — integers, booleans, unit. A *memory* (we will also say *heap*) is a total function from addresses to values: $\mathsf{State} := \mathsf{Loc} \to \mathsf{Val}$.

So a memory σ is a function. To ask what is in cell 7 you write $\sigma\,7$. Three features of this choice matter later, and it is worth saying now which.

It is total — defined at every address. There is no unallocated cell, no null, no crash on a bad read: $\sigma\,999999$ is perfectly well defined and holds whatever is there. We therefore never carry a definedness side condition, and ownership of memory is expressed by *footprints* (§3.2) rather than by a heap's domain. Programs in the surface language do declare buffers and end their lifetimes, but that happens in the compiler and leaves no trace in the program that runs (§4.1).

Addresses are numbers, not an opaque type, so a question like "is $i \cdot n + j$ inside the block $[b,\ b + \ell)$?" is ordinary linear arithmetic and a machine can decide it. Richer value types are encoded into Val through a type class, never by adding constructors to Definition 1.

Updating it builds a new function.

Definition 2 (Update)**.** For a memory σ, an address l and a value v,

$$\sigma[l \mapsto v] \;:=\; \lambda k.\ \text{if } k = l \text{ then } v \text{ else } \sigma\,k.$$

That is, $\sigma[l \mapsto v]$ is "the same as σ, except cell l now holds v". Two consequences are immediate from Definition 2, and we will use them constantly:

$$\sigma[l \mapsto v]\,l = v \tag{1}$$

$$\sigma[l \mapsto v]\,k = \sigma\,k \qquad \text{whenever } k \neq l \tag{2}$$

Remark 3. Equation (2) is the single load-bearing fact in this document. Everything below is machinery for lifting it from *one cell* to *whole programs*: Lemma 32 lifts it to one program, Lemma 34 to a program observed on two memories, Lemma 35 to two sibling branches, and Lemma 57 to a single step of an arbitrary interleaving. If you keep one thing in mind while reading, keep this: a write to cell l cannot be seen at cell k.

2.2 Programs

Now: what is a program? Here is the design choice everything rests on. **A program is a piece of data** — a finite tree — not a Lean function.

Why? Because we want to ask questions like "which cells does this program write?". You cannot ask that of an opaque function; you can ask it of a tree. And because if parallel composition were a Lean function, Lean's own equational theory would already fix its meaning, and there would be nothing left to prove (§2.4 makes that concrete).

A fork has to hold several programs at once, and those programs need not return the same type: one child might compute an `Int` while its sibling computes a `Bool`. An ordinary `List` (`ParM Int`) cannot hold both, so the children of a fork get a type of their own, indexed by the list of types they return.

Definition 4 (Children of a fork)**.**

```
def Results : List Type -> Type
  | []      => Unit
  | a :: as => a * Results as

inductive Childs (Node : Type -> Type 1) : List Type -> Type 1 where
  | nil  : Childs Node []
  | cons : Node a -> Childs Node as -> Childs Node (a :: as)
```

Read both at the index `[Int, Bool]`. A value of `Childs ParM [Int, Bool]` is built by `cons`, `cons`, `nil` and holds exactly two programs, a `ParM Int` and a `ParM Bool`; no other value of that type exists. Meanwhile `Results [Int, Bool]` unfolds to `Int` × (`Bool` × `Unit`) — one factor per child, with the `Unit` left over from the empty case. The index is thus a static record of how many children a fork has and what each returns, so the continuation can be handed a tuple of exactly the right shape.

Definition 5 (Programs)**.**

```
inductive ParM : Type -> Type 1 where
  | pure  : a -> ParM a
  | read  : Loc -> (Val -> ParM a) -> ParM a
  | write : Loc -> Val -> (Unit -> ParM a) -> ParM a
  | par   : Childs ParM as -> (Results as -> ParM b) -> ParM b
```

In words, the four node shapes are:

`pure` a	stop; the answer is a
`read` l k	read cell l, then continue as k applied to the value you got
`write` l v k	write v into cell l, then continue as k applied to ()
`par` cs k	run everything in cs at the same time, then continue as k applied to all their answers

Continuations sit inside the constructors. The odd-looking part is k. In `read` l k, the thing k is a *function*: it takes the value that was read and returns the rest of the program. It has to be a function, because what happens next genuinely depends on what was read — if cell 7 holds 3 you might go one way, and if it holds 4, another. So `read` does not "return a value"; it stores the remainder of the program as a function of the value. This is called the *freer* presentation of a monad, and k is the *continuation*.

A program is therefore a finite tree whose root is always the next action to perform and whose leaves are all `pure`.

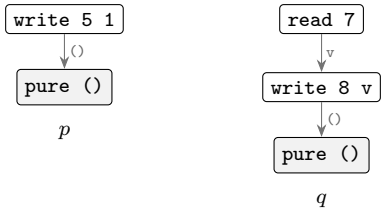
Example 6. The two jobs from §1, written out. "Write 1 into cell 5" is

$$p \;:=\; \texttt{write}\ 5\ 1\ (\lambda_.\ \texttt{pure}\ ())$$

and "copy cell 7 into cell 8" is

$$q \;:=\; \texttt{read}\ 7\ (\lambda v.\ \texttt{write}\ 8\ v\ (\lambda_.\ \texttt{pure}\ ())).$$

As trees, growing downward:



We will carry p and q through the whole document.

Sequential composition is not a constructor. `bind` p f walks the tree of p and replaces every leaf `pure` a with the tree $f\,a$ (Figure 1):

$$\begin{aligned}
\texttt{bind}\ (\texttt{pure}\ a)\ f &= f\,a \\
\texttt{bind}\ (\texttt{read}\ l\ k)\ f &= \texttt{read}\ l\ (\lambda v.\ \texttt{bind}\ (k\,v)\ f) \\
\texttt{bind}\ (\texttt{write}\ l\ v\ k)\ f &= \texttt{write}\ l\ v\ (\lambda u.\ \texttt{bind}\ (k\,u)\ f) \\
\texttt{bind}\ (\texttt{par}\ cs\ k)\ f &= \texttt{par}\ cs\ (\lambda vs.\ \texttt{bind}\ (k\,vs)\ f)
\end{aligned}$$

Because the composite is an actual tree rather than a node recording that two trees were composed, the monad laws are equalities *of trees* and need no quotient. Left identity `bind` (`pure` a) $f = f\,a$ is one unfolding of the definition. Right identity and associativity take one induction each. Had `bind` been a constructor, `bind` (`bind` p f) g and `bind` p ($\lambda a.$ `bind` ($f\,a$) g) would be two distinct trees that we would have to identify by hand.

The same property removes evaluation contexts from the operational semantics of §3.6: a small-step rule normally has to locate the redex somewhere inside the term, and here it is always at the root.

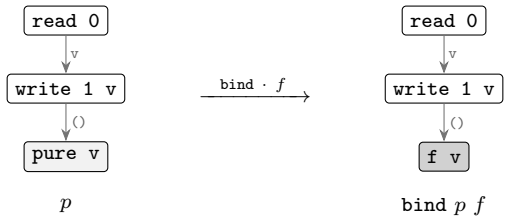


Figure 1: $p =$ `read` 0 ($\lambda v.$ `write` 1 v ($\lambda_.$ `pure` v)), drawn as a tree. Each node stores the rest of the program as a function of what its own effect returns, and each edge is labelled with the value passed on. `bind` p f grafts f onto the `pure` leaf and changes nothing else.

The fork is N-ary and heterogeneous. A `par` node holds one spine of n children and one continuation receiving all n results together. Binary parallelism is the case $n = 2$: it is not a second constructor, and an n-child fork is not a tower of nested binary forks. Every theorem below is proved at one fork with n children. Two consequences recur: the race check at a fork compares all $\binom{n}{2}$ pairs at once rather than spreading them over nesting levels, and a round of the cost semantics (§6) advances all n children together.

Every program terminates. `ParM` is an *inductive* type, so each of its values is a *finite* tree and a non-terminating loop cannot be written down at all. Lean 4 has no coinductive types, so this is a boundary of the approach rather than an unfinished feature. Every example below is structurally recursive or fuel-bounded with the fuel proved sufficient. This restriction is not a nuisance we tolerate; it is what licenses Definition 9 to be a *function*, and Definition 9 is the whole proof strategy.

Definition 7 (The two sides of the theorem)**.** Write $\Pi[\,p\;q\,]$ for `cons` p (`cons` q `nil`). Then

$$\begin{aligned}
\mathsf{parPair}\ p\ q &\coloneqq \texttt{par}\ \Pi[\,p\;q\,]\ (\lambda r.\ \texttt{pure}\ (r.1, r.2.1))\\
\mathsf{seqPair}\ p\ q &\coloneqq \texttt{bind}\ p\ (\lambda a.\ \texttt{bind}\ q\ (\lambda b.\ \texttt{pure}\ (a, b)))\\
\mathsf{parAll}\ ps &\coloneqq \texttt{par}\ (\mathsf{childsOf}\ ps)\ (\lambda x.\ \texttt{pure}\ (\mathsf{resultsToList}\ ps\ x))\\
\mathsf{seqAll}\ [\,] &\coloneqq \texttt{pure}\ [\,]\\
\mathsf{seqAll}\ (p :: ps) &\coloneqq \texttt{bind}\ (\mathsf{seqPair}\ p\ (\mathsf{seqAll}\ ps))\ (\lambda r.\ \texttt{pure}\ (r.1 :: r.2))
\end{aligned}$$

Here childsOf turns a `List` (`ParM` α) into the spine `Childs ParM` $[\alpha, \ldots, \alpha]$ at the index repeating α once per element, and resultsToList is its inverse on results, flattening the nested pairs of Results back into a list. They are what let a homogeneous list be forked even though `par` is heterogeneous.

Note the asymmetry, because it is the shape of the theorem. parAll ps on a list of n programs is *one* `par` node with n children. seqAll ps is built from `bind` alone and contains *no* `par` node whatsoever. These are the two sides the equivalence theorem compares, and parPair/seqPair are their two-child instances.

2.3 Induction over a nested inductive

Lean's `induction` tactic does not work on `ParM`:

```
The 'induction' tactic does not support the type 'ParM' because it is a
nested inductive type
```

This is the largest tax the representation imposes. It is paid once, by proving two eliminators that every later proof uses, and the reason the tactic gives up determines the shape of those two — so it is worth a paragraph.

What a motive is. To prove $\forall p,\ P\,p$ by induction you must first say what is being proved about an arbitrary p, *as a function of* p. That function — here P, of type `ParM` $\alpha \to$ `Prop` — is called the *motive*. A recursor takes the motive first, then asks for one case per constructor, each case free to assume the motive at every smaller program, and returns the motive at the program it was handed. The `induction` tactic is a wrapper: it recovers the motive from the goal by abstracting the goal over the variable being inducted on, then applies the recursor.

Why recovering it from the goal is not enough here. `ParM` is defined in terms of `Childs ParM`, so Lean generates a recursor that proves two things at once and takes *two* motives:

```
ParM.rec : {motive_1 : (a : Type) -> ParM a -> Sort u} ->
           {motive_2 : (as : List Type) -> Childs ParM as -> Sort u} ->
           (four cases for ParM, then two for Childs) ->
           (p : ParM a) -> motive_1 a p
```

A goal $P\,p$ fixes $motive_1$ and says *nothing at all* about $motive_2$: it mentions a program and never mentions a spine of children, so there is nothing for the tactic to abstract. Choosing $motive_2$ is therefore a genuine decision, and two choices are useful.

Proposition 8 (Induction principles)**.** *Let* $motive : (\alpha : \texttt{Type}) \to \texttt{ParM}\ \alpha \to \texttt{Prop}$.

(i) ***Single-motive induction*** *proves* $\forall \alpha\ (p : \texttt{ParM}\ \alpha)$, *motive* $\alpha\ p$ *from four cases, one per constructor. Its* `par` *case may assume the motive of the continuation at every tuple of results, and the motive of every child, the latter packaged as* All *motive cs — an iterated conjunction defined by* All *motive* `nil` $=$ `True` *and* All *motive* (`cons` $p\ ps$) $=$ *motive* $p \wedge$ All *motive* ps.

(ii) ***Mutual induction*** *takes a second motive* $motive_2 : (\alpha s : \texttt{List Type}) \to \texttt{Childs ParM}\ \alpha s \to \texttt{Prop}$ *of its own and proves the* pair

$$\big(\forall \alpha\ (p : \texttt{ParM}\ \alpha),\ motive_1\ \alpha\ p\big)\ \wedge\ \big(\forall \alpha s\ (cs : \texttt{Childs ParM}\ \alpha s),\ motive_2\ \alpha s\ cs\big)$$

from six cases: the four above, with the `par` *case now assuming* $motive_2\ \alpha s\ cs$*, plus* $motive_2$ *of the empty spine, plus a* `cons` *case deriving* $motive_2\ (\alpha :: \alpha s)$ *from* $motive_1\ \alpha$ *for the head and* $motive_2\ \alpha s$ *for the tail.*

Proof. Both pick $motive_2$ in the generated recursor and then discharge its two `Childs` cases themselves, so that what reaches the caller is only the four `ParM` cases.

For (i), take $motive_2\ cs :=$ All *motive cs*. The `nil` case then asks for All *motive* `nil`, which unfolds to `True`. The `cons` case is handed *motive* of the head and All *motive* of the tail, and must produce All *motive* (`cons` *p ps*), which unfolds to the conjunction of exactly those two. Both obligations are immediate, and that is the whole content of the choice: it makes the children's statement be the program's statement, repeated.

For (ii), pass $motive_2$ through unchanged; the recursor gives the first component directly. The second component is a separate, ordinary induction on *cs*, which is legal because `Childs` is an *unnested* inductive: its `nil` case is the hypothesis for the empty spine, and its `cons` case applies the `cons` hypothesis to the first component at the head and to the inductive hypothesis at the tail. □

The moral. Which eliminator a proof needs depends on whether the claim about a spine of children is the *same* claim as the one about a program. For `bind` it is: `bind` leaves a fork's children untouched and only extends its continuation, so (i) suffices. For execution it is not: runSeq acts on a program while runChilds acts on a spine, and they are different functions, so the children-level claim must be stated separately and the two proved together by (ii).

Every proof in §3 and §5 is of the second kind: each is stated as a conjunction of a program-level and a children-level claim, and in each the `cons` case assumes the claim for the head child and for the rest of the spine and concludes it for the whole spine. We call that case the **2-to-N step** throughout. It is the same argument every time, which is why the N-ary development is no longer than a binary one would have been.

2.4 Why not Lean's `Task`

Lean provides `Task` for genuine runtime parallelism, and it is tempting to state the theorem about that. The attempt is not merely inconvenient; it is *vacuous*. `Task` is a one-field structure whose body is the value, so

$$(\texttt{Task.spawn}\ f).\texttt{get}\ =\ f\ ()$$

holds by `rfl` — by unfolding definitions. The parallel program and the sequential one are then *the same term*, and any equivalence between them is an identity with no content. The hypothesis could be deleted and the theorem would still hold.

The general principle: **the meaning of `par` must be supplied by us, and must be genuinely interleaving, or there is nothing to prove.** That rules out a shallow embedding, where `par` would be a Lean function and Lean's equational theory would fix its meaning; it equally rules out treating a compiled runtime as the semantics. It does *not* require deep-embedding the whole language: binders, data types, arithmetic and recursion are all inherited from Lean, and only the four constructs whose interleaving behaviour is at issue get reified. `Task` remains the compilation *target*, and nothing is ever proved there.

The moral. Because a program is data, one tree can be read in many ways, and the development is organised around exactly that: step is the operational semantics (§3.6), runSeq the canonical

run (§3.1), accesses the footprint (§3.2), work and pstep the cost measures (§6), and there is a compiler to `Task`. A shallow embedding gives you only the first.

3 What a program touches, and the machine that runs it

This section defines the two sides of the main theorem: the *hypothesis* you discharge per program, and the *relation* the conclusion is stated over. It closes with four locality lemmas that connect them.

3.1 The reference run

Here is the first interpreter, and it is deliberately boring. It takes a program and a memory and returns the answer and the final memory. **It contains no parallelism at all**: at a `par` node it runs child 1 to completion, then child 2 from where child 1 stopped, then child 3, and so on.

Definition 9 (The canonical run)**.**

$$\begin{aligned}
\mathsf{runSeq}\ (\texttt{pure}\ a)\ \sigma &= (a, \sigma) \\
\mathsf{runSeq}\ (\texttt{read}\ l\ k)\ \sigma &= \mathsf{runSeq}\ (k\,(\sigma\, l))\ \sigma \\
\mathsf{runSeq}\ (\texttt{write}\ l\ v\ k)\ \sigma &= \mathsf{runSeq}\ (k\,())\ \sigma[l \mapsto v] \\
\mathsf{runSeq}\ (\texttt{par}\ cs\ k)\ \sigma &= \mathsf{runSeq}\ (k\, vs)\ \tau \qquad \text{where } (vs, \tau) = \mathsf{runChilds}\ cs\ \sigma
\end{aligned}$$

and the children run strictly left to right, each from the memory the previous one left:

$$\begin{aligned}
\mathsf{runChilds}\ [\,]\ \sigma &= ((), \sigma) \\
\mathsf{runChilds}\ (p :: ps)\ \sigma &= ((a, ws), \tau_2) \quad \text{where } (a, \tau_1) = \mathsf{runSeq}\ p\ \sigma \text{ and } (ws, \tau_2) = \mathsf{runChilds}\ ps\ \tau_1
\end{aligned}$$

Why is this allowed to be a function rather than a relation? Because programs are finite trees (§2.2), so the recursion always bottoms out: every case recurses into a strictly smaller tree, or into a continuation applied to a value already in hand. This is plain structural recursion, with no cleverness and no termination argument to supply.

Remark 10 (Why this is the whole strategy)*.* runSeq is the *reference answer*. It is deterministic, total and computable. The main theorem will say: the real parallel machine, under every schedule whatsoever, agrees with runSeq. We will never compare the parallel program to the sequential one directly. We will show both of them equal this third thing.

Example 11 (The reference run, by hand)*.* Let σ_0 be the all-zeros memory, $\sigma_0\, l = 0$ for every l, and take p, q from Example 6. Running p:

$$\begin{aligned}
\mathsf{runSeq}\ (\texttt{write}\ 5\ 1\ (\lambda_.\ \texttt{pure}\ ()))\ \sigma_0 &= \mathsf{runSeq}\ (\texttt{pure}\ ())\ \sigma_0[5 \mapsto 1] && \text{(the write rule)} \\
&= ((),\ \sigma_0[5 \mapsto 1]) && \text{(the pure rule)}
\end{aligned}$$

Write $\sigma_1 \coloneqq \sigma_0[5 \mapsto 1]$. Running q from σ_1:

$$\mathsf{runSeq}\ (\texttt{read}\ 7\ (\lambda v.\ \texttt{write}\ 8\ v\ \cdots))\ \sigma_1 = \mathsf{runSeq}\ (\texttt{write}\ 8\ (\sigma_1\, 7)\ \cdots)\ \sigma_1 \quad \text{(the read rule)}$$

and what is $\sigma_1\, 7$? By (2), since $7 \neq 5$, it is $\sigma_0\, 7 = 0$. So the run continues to $((),\ \sigma_1[8 \mapsto 0])$.

Now the fork. By Definition 9, $\mathsf{runChilds}\ \Pi[p\ q\,]\ \sigma_0$ runs p from σ_0, reaching σ_1, then q from σ_1, reaching $\sigma_1[8 \mapsto 0]$. Hence

$$\mathsf{runSeq}\ (\mathsf{parPair}\ p\ q)\ \sigma_0 = \big(((), ()),\ \sigma_0[5 \mapsto 1][8 \mapsto 0]\big).$$

Cell 5 holds 1 and cell 8 holds 0. Keep that memory in mind; we will meet it again in Example 37 and §5.8.

3.2 Footprints

To state the condition under which parallel equals sequential, we must talk about *which cells* a program touches. So: a second interpreter, walking the same tree, collecting addresses instead of computing answers.

Definition 12 (Footprints)**.** A Footprint is a pair of lists of addresses, the cells read and the cells written, written $f = (R(f), W(f))$, with

$$f \cup g \coloneqq (R(f) +\!\!+ R(g),\ W(f) +\!\!+ W(g)), \qquad \mathsf{empty} \coloneqq ([\,], [\,]).$$

Lists rather than finite sets, because every question we ask of a footprint is a membership test or a disjointness test. Duplicates are therefore harmless, and a concrete footprint stays computable. Union is associative with empty as a unit, so folding footprints needs no bracketing convention.

Definition 13 (The footprint of a program)**.**

$$\begin{aligned}
\mathsf{accesses}\ (\mathtt{pure}\ a)\ \sigma &= \mathsf{empty} \\
\mathsf{accesses}\ (\mathtt{read}\ l\ k)\ \sigma &= ([l], [\,]) \cup \mathsf{accesses}\ (k\,(\sigma\, l))\ \sigma \\
\mathsf{accesses}\ (\mathtt{write}\ l\ v\ k)\ \sigma &= ([\,], [l]) \cup \mathsf{accesses}\ (k\,())\ \sigma[l \mapsto v] \\
\mathsf{accesses}\ (\mathtt{par}\ cs\ k)\ \sigma &= \mathsf{accChilds}\ cs\ \sigma \cup \mathsf{accesses}\ (k\, vs)\ \tau \qquad (vs, \tau) = \mathsf{runChilds}\ cs\ \sigma \\
\mathsf{accChilds}\ [\,]\ \sigma &= \mathsf{empty} \\
\mathsf{accChilds}\ (p :: ps)\ \sigma &= \mathsf{accesses}\ p\ \sigma \cup \mathsf{accChilds}\ ps\ (\mathsf{runSeq}\ p\ \sigma).2
\end{aligned}$$

Why a footprint needs a memory argument. This confuses every reader once, so let us be careful. Look at the `read` clause. To see what happens *after* the read we need k applied to something, and k is a function expecting the value read — so we must supply $\sigma\, l$. There is no way to look past a `read` without saying what it returned. **A program's footprint is a function of the memory it starts in.** That is not a defect of the definition; it is forced by continuations, and it is why accesses is a fold over the same tree runSeq walks rather than a syntactic function.

Concretely, `read` 7 (λv. `write` v 1 $\cdots$) writes to a cell whose *address* comes out of cell 7. Change the starting memory and it writes somewhere else. §4.2 is about recovering a syntactic bound anyway, at a stated cost in precision.

Example 14 (Footprints, by hand). Continuing Example 11,

$$\begin{aligned}
&\mathsf{accesses}\ p\ \sigma_0 = ([\,], [5]) \cup \mathsf{accesses}\ (\mathtt{pure}\ ())\ \sigma_1 = ([\,], [5]) \cup \mathsf{empty} = ([\,], [5]) \\
&\mathsf{accesses}\ q\ \sigma_1 = ([7], [\,]) \cup \big(([\,], [8]) \cup \mathsf{empty}\big) = ([7], [8])
\end{aligned}$$

So p reads nothing and writes cell 5; q reads cell 7 and writes cell 8. Exactly what one would say by eye — the definition is doing the right thing.

3.3 The staggered measurement

Definition 15 (Per-child footprints)**.**

$$\begin{aligned}
\mathsf{childFootprints}\ [\,]\ \sigma &= [\,] \\
\mathsf{childFootprints}\ (p :: ps)\ \sigma &= \mathsf{accesses}\ p\ \sigma :: \mathsf{childFootprints}\ ps\ \tau_1, \qquad (_, \tau_1) = \mathsf{runSeq}\ p\ \sigma
\end{aligned}$$

This keeps one footprint *per child*, where accChilds merged them all into one. The distinction matters: accChilds is what you want when a fork is treated as a single unit, and childFootprints is what the all-pairs independence check ranges over.

Remark 16 (The crux of the entire theorem). Look at *where* each child is measured. Child 1 is measured at σ. Child 2 is measured at $(\mathsf{runSeq}\ p\ \sigma).2$ — the memory **after child 1 has completely finished**. Child 3 after children 1 and 2 have finished. And so on.

But the real parallel machine will not run child 2 at that memory. It may run child 2 first, at σ. So we are about to define our hypothesis using one memory while the machine uses a different one, and nothing written so far says the two agree.

This gap is the whole technical content of the theorem. It is closed, once, in Lemma 35, and the one place it is *used* is sub-case B of Lemma 57. Every other case of every other proof is bookkeeping.

For our running example,

$$\mathsf{childFootprints}\ \Pi[\,p\ q\,]\ \sigma_0 = [\ ([\,],[5]),\ ([7],[8])\ ],$$

and the second entry was measured at σ_1, not at σ_0.

3.4 Independence

Definition 17 (Bernstein independence)**.** $\mathsf{Indep}\ f_1\ f_2$ holds when all three of these sets are empty:

$$\text{(C1)}\ \ W(f_1)\cap W(f_2)=\emptyset, \qquad \text{(C2)}\ \ W(f_1)\cap R(f_2)=\emptyset, \qquad \text{(C3)}\ \ W(f_2)\cap R(f_1)=\emptyset.$$

Three clauses, and the reader must notice *which one is missing*: there is no clause $R(f_1)\cap R(f_2)=\emptyset$. Two computations may read the same cell as much as they like, because reading changes nothing and nobody can tell who read first.

That omission is load-bearing rather than a convenience. Matrix multiplication computes each output cell $C[i,j]$ in its own thread; the threads write disjoint cells, so (C1)–(C3) hold, but every thread in row i reads the whole of row i of A. A criterion with a fourth clause on reads would reject matrix multiplication, and with it most of the programs anyone wants to parallelise. (A separation-logic $*$ is exactly such a criterion, which is the subject of the paper's related-work section.)

Example 18 (Independent, and not). For p and q of Example 6, with $f_1 = ([\,],[5])$ and $f_2 = ([7],[8])$:

$$\text{(C1)}\ \{5\}\cap\{8\}=\emptyset\ \checkmark \qquad \text{(C2)}\ \{5\}\cap\{7\}=\emptyset\ \checkmark \qquad \text{(C3)}\ \{8\}\cap\emptyset=\emptyset\ \checkmark$$

so $\mathsf{Indep}\ f_1\ f_2$ holds.

Now replace q by the broken sibling of §1, $q' \coloneqq \texttt{read}\ 5\ (\lambda v.\ \texttt{write}\ 8\ v\ \cdots)$, which copies cell 5 instead of cell 7. Then $\mathsf{accesses}\ q'\ \sigma_1 = ([5],[8])$ and

$$\text{(C2)}\ W(f_1)\cap R(f_2)=\{5\}\cap\{5\}=\{5\}\neq\emptyset\ \times$$

so independence fails. This is precisely the program whose two schedules disagreed. The condition catches it, and clause (C2) is the one that does.

Definition 19 (The N-ary obligation)**.** $\mathsf{IndepAll}\ [\,] = \texttt{True}$ and $\mathsf{IndepAll}\ (f :: fs) = \big(\forall g \in fs,\ \mathsf{Indep}\ f\ g\big) \wedge \mathsf{IndepAll}\ fs$.

Unfolded, this is pairwise independence of all $\binom{n}{2}$ pairs, stated flatly rather than spread across nesting levels, and $\mathsf{IndepAll}\ [f_1,f_2]$ is just $\mathsf{Indep}\ f_1\ f_2$.

Proposition 20 (Everything here is decidable)**.** *There are Boolean functions* $\mathsf{disjointB}$, indepB, $\mathsf{indepAllB}$ *with* $\mathsf{indepB}\ f_1\ f_2 = \texttt{true} \leftrightarrow \mathsf{Indep}\ f_1\ f_2$ *and* $\mathsf{indepAllB}\ fs = \texttt{true} \leftrightarrow \mathsf{IndepAll}\ fs$.

Proof. Take $\mathsf{disjointB}\ a\ b \coloneqq a.\mathsf{all}\ (\lambda l.\ \neg\, b.\mathsf{contains}\ l)$, which decides disjointness of two lists because a universally quantified Boolean test over a list is equivalent to its all. Then indepB is the conjunction of three such checks, matching Definition 17 clause for clause, and $\mathsf{indepAllB}$ follows by induction on the list. □

This is what makes the hypothesis cheap in practice: for a concrete program and memory the whole obligation is discharged by evaluation, often by `rfl`.

3.5 Well-formedness: the hypothesis

Definition 21 (Well-formedness)**.**

$$\begin{aligned}
\mathsf{WellFormed}\ (\mathtt{pure}\ a)\ \sigma &= \mathtt{True}\\
\mathsf{WellFormed}\ (\mathtt{read}\ l\ k)\ \sigma &= \mathsf{WellFormed}\ (k\,(\sigma\, l))\ \sigma\\
\mathsf{WellFormed}\ (\mathtt{write}\ l\ v\ k)\ \sigma &= \mathsf{WellFormed}\ (k\,())\ \sigma[l \mapsto v]\\
\mathsf{WellFormed}\ (\mathtt{par}\ cs\ k)\ \sigma &= \mathsf{IndepAll}\ (\mathsf{childFootprints}\ cs\ \sigma)\\
&\qquad \wedge\ \mathsf{wfChilds}\ cs\ \sigma\ \wedge\ \mathsf{WellFormed}\ (k\, vs)\ \tau\\
\mathsf{wfChilds}\ [\,]\ \sigma &= \mathtt{True}\\
\mathsf{wfChilds}\ (p :: ps)\ \sigma &= \mathsf{WellFormed}\ p\ \sigma\ \wedge\ \mathsf{wfChilds}\ ps\ \tau_1
\end{aligned}$$

where $(vs, \tau) = \mathsf{runChilds}\ cs\ \sigma$ in the fork clause and $(_, \tau_1) = \mathsf{runSeq}\ p\ \sigma$ in the children clause.

In words: **walk the whole tree along the canonical run, and at every `par` node you meet, check that all pairs of its children are independent at the memory that node sees.** The three conjuncts at a fork each do a job: the children do not interfere with each other; each child is itself well-formed, since it may contain forks of its own; and whatever follows the join is well-formed too.

Why not just check the outermost fork? Because a child can race *with itself* while the two children stay disjoint from each other. Take

$$\mathtt{par}\ \Pi[\ \underbrace{\mathtt{par}\ \Pi[\,\mathtt{write}\ 3\ \ \mathtt{write}\ 3\,]}_{\text{child 1}}\ \ \underbrace{\mathtt{write}\ 99}_{\text{child 2}}\ ].$$

Child 1's total footprint is $\{3\}$ and child 2's is $\{99\}$: disjoint, so an outermost-only check passes. But child 1 is a race. Hence the recursion in Definition 21. This is the only obligation discharged per program; everything in §5 is proved once and for all.

Example 22. $\mathsf{WellFormed}\ (\mathsf{parPair}\ p\ q)\ \sigma_0$ unfolds to three conjuncts: $\mathsf{IndepAll}\ [([\,], [5]), ([7], [8])]$, which holds by Example 18; $\mathsf{WellFormed}\ p\ \sigma_0 \wedge \mathsf{WellFormed}\ q\ \sigma_1 \wedge \mathtt{True}$, all trivially true since neither contains a `par` node; and $\mathsf{WellFormed}$ of a `pure` leaf, also trivial. So the running example is well-formed.

3.6 The real machine

Everything so far was reference material. **Now we define the machine that is actually allowed to interleave**, and it is this machine the theorem is about.

First we need to detect races: two threads about to touch one cell with at least one write.

Definition 23 (Labels and conflict)**.** A Label is `read` l or `write` l. Two labels *conflict* when they name the same address and at least one is a write. Values are irrelevant to races and are not recorded.

Definition 24 (Ready actions and the race test)**.** nextAccs p collects the actions p could perform *right now*:

$$\mathsf{nextAccs}\ (\mathtt{pure}\ a) = [\,], \quad \mathsf{nextAccs}\ (\mathtt{read}\ l\ k) = [\mathtt{read}\ l], \quad \mathsf{nextAccs}\ (\mathtt{write}\ l\ v\ k) = [\mathtt{write}\ l],$$

and nextAccs (par cs k) is all the children's ready actions flattened, so a deeply nested program contributes every one of its live heads and no pair is missed. nextAccsChilds cs keeps one list *per* child. Then racing as bs holds when some action in as conflicts with some action in bs, and racingAll holds when some *two distinct* children's lists race.

Keeping one list per child, rather than one flat list, is what lets racingAll compare pairs of children while ignoring a child's conflicts with itself. One thread about to write cell 3 and later write it again is not a race; two threads both about to write cell 3 are.

Definition 25 (All children finished)**.** allPure cs is some vs when every child is a pure leaf, collecting their values into the tuple the continuation expects, and none as soon as one child still has work left.

Definition 26 (Configurations and one step)**.** A configuration is run p σ or **wrong**. The step function returns the *list* of configurations we could move to:

$$\begin{aligned}
\mathsf{step}\ (\mathtt{pure}\ a)\ \sigma &= [\,] \\
\mathsf{step}\ (\mathtt{read}\ l\ k)\ \sigma &= [\ \mathtt{run}\ (k\,(\sigma\,l))\ \sigma\ ] \\
\mathsf{step}\ (\mathtt{write}\ l\ v\ k)\ \sigma &= [\ \mathtt{run}\ (k\,())\ \sigma[l \mapsto v]\ ] \\
\mathsf{step}\ (\mathtt{par}\ cs\ k)\ \sigma &= \begin{cases} [\ \mathbf{wrong}\ ] & \text{if racingAll (nextAccsChilds } cs) \\ [\ \mathtt{run}\ (k\,vs)\ \sigma\ ] & \text{else if allPure } cs = \mathsf{some}\ vs \\ \text{one successor per ready child} & \text{otherwise, via stepChilds} \end{cases}
\end{aligned}$$

Here stepChilds cs σ advances *any one* child and rebuilds the spine around it; it is type-preserving, and a child aborting is inherited by the whole fork. Finally stepC lifts step to configurations, with stepC **wrong** $= [\,]$.

The list *is* the nondeterminism. This is the key idea of the semantics, and it is why no schedulers or thread identifiers appear anywhere. A par node with three children that can all move returns a list of *three* configurations, and each is a different choice the machine could make. All possible interleavings are exactly all the paths you can trace through these lists, and nothing outside the list counts as a schedule.

The par case is a trichotomy. Exactly one of the three branches fires (Figure 2). The second and third cannot both apply, because if every child is a finished leaf then no child can move:

Proposition 27 (Join and step are exclusive)**.** *If* allPure cs *is defined then* stepChilds cs $\sigma = [\,]$.

Proof. Induction on cs. The nil case is immediate. The cons case: for allPure to be defined the head must be pure a, whose step is the empty list, and the tail is handled by the induction hypothesis; a concatenation of two empty lists is empty. □

wrong is a black hole. It has no successors, so it is terminal and absorbing. A racy program is not assigned a behaviour that we then have to reason about — it falls off the edge of the world and is *excluded*. This is why we never argue about whether memory operations are atomic: a racy program has no outcomes at all, so the question never arises. The race test moreover runs at *every* step, not only at the start, so a run that never reaches **wrong** is race-free at every instant.

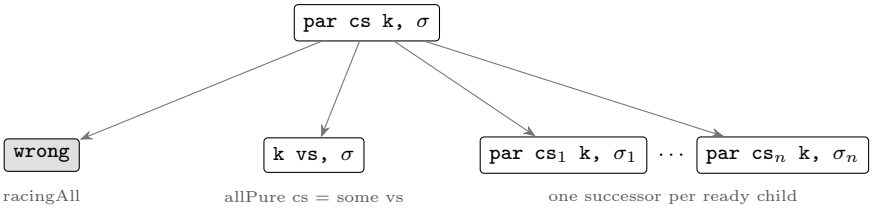


Figure 2: The par case of step. The three branches are tried in this order and are mutually exclusive (Proposition 27). Only the third produces more than one successor, and that is the only place a schedule is ever chosen.

Example 28 (A step with two successors). Take parPair $p\ q$ at σ_0. We have nextAccs $p = [\texttt{write}\ 5]$ and nextAccs $q = [\texttt{read}\ 7]$. Different cells, so no conflict, so racingAll is false — branch one does not fire. Is allPure defined? No: p is a `write` node, not a leaf — branch two does not fire. So branch three gives a list of *two* configurations:

$$\left[\ \texttt{run}\ (\texttt{par}\ \Pi[\,\texttt{pure}\ ()\ \ q\,]\ k)\ \sigma_0[5 \mapsto 1],\quad \texttt{run}\ (\texttt{par}\ \Pi[\,p\ \ \texttt{write}\ 8\ 0\,]\ k)\ \sigma_0\ \right]$$

— one in which p went first, one in which q went first. Those are the two schedules, and the theorem must hold down both of them, and down every branch of every branch.

Change q to q' (which reads cell 5) and the same call returns the single successor [**wrong**]: now `write` 5 and `read` 5 conflict.

Definition 29 (Reachability)**.** Reaches is the reflexive–transitive closure of stepC, generated by Reaches $c\ c$ and, from Reaches $c\ c'$ together with $c'' \in$ stepC c', Reaches $c\ c''$. We write Safe c for $\neg$ Reaches c **wrong**. The children-level closure ReachesChilds is defined the same way from stepChilds, relating one pair (cs, σ) to another with nobody having aborted.

Reaches is the object every statement about "all interleavings" is really about. **We never quantify over schedules explicitly**: a schedule *is* a path through the closure, so the closure does the quantification for us. ReachesChilds replaces the left-branch and right-branch lemmas a binary development would need — with n children there is no left and right, only "some child moved".

Proposition 30 (Basic properties of Reaches)**.** Reaches *is transitive; a configuration with no successors reaches only itself; and if* stepC $c = [c_0]$ *then* Reaches $c\ c'$ *implies* $c' = c$ *or* Reaches $c_0\ c'$. ReachesChilds *is transitive.*

Proof. Each by induction on the closure. The third is the deterministic-step case analysis, used for `read` and `write`, whose successor lists are singletons: either the run stopped where it was, or its first move went to the unique successor. □

3.7 What we want to prove

Definition 31 (Outcome equivalence)**.** Programs p and q are *outcome-equivalent from* σ, written $p \parallel\!\equiv q$, when for every answer r and every final memory σ',

$$\text{Reaches}\ \langle p, \sigma\rangle\ \langle \texttt{pure}\ r, \sigma'\rangle\ \longleftrightarrow\ \text{Reaches}\ \langle q, \sigma\rangle\ \langle \texttt{pure}\ r, \sigma'\rangle.$$

That is: exactly the same set of possible finishing states. The goal of §5 is

$$\text{WellFormed}\ (\text{parPair}\ p\ q)\ \sigma\ \Longrightarrow\ \text{parPair}\ p\ q \parallel\!\equiv \text{seqPair}\ p\ q\ \text{from}\ \sigma.$$

3.8 Locality: four lemmas

A computation cannot observe what it does not touch. The four lemmas below make that precise in four steps — a program does not change cells outside its writes (Lemma 32); it computes the same thing on two memories that agree wherever it looks (Lemma 34); so an independent neighbour's run is invisible to it (Lemma 35); and therefore the two can never race (Lemma 39). All four are about footprints and the canonical run, with no mention of interleaving anywhere. §5 uses them in exactly that order.

3.8.1 Frame

Lemma 32 (Frame)**.** *If* $l \notin W(\mathsf{accesses}\ p\ \sigma)$ *then* $(\mathsf{runSeq}\ p\ \sigma).2\ l = \sigma\ l$*, and likewise for* runChilds *against* accChilds*.*

Scratch work. The claim is "a program leaves alone what it does not write". For a single `write` l' node this is exactly (2). So the proof should be an induction whose only non-trivial case is `write`, and in that case the hypothesis should hand us $l \neq l'$. Let us check that it does.

Proof. By mutual induction on p (Proposition 8(ii)). Six cases.

Case `pure` a. Here $\mathsf{runSeq}\ (\mathtt{pure}\ a)\ \sigma = (a, \sigma)$, so the final memory *is* σ and the claim reads $\sigma\ l = \sigma\ l$.

Case `read` $l'\ k$. By Definition 13, $\mathsf{accesses}\ (\mathtt{read}\ l'\ k)\ \sigma = ([l'], [\,]) \cup \mathsf{accesses}\ (k\,(\sigma\, l'))\ \sigma$. Taking writes of both sides, the first summand contributes nothing, so $W(\mathsf{accesses}\ (\mathtt{read}\ l'\ k)\ \sigma) = W(\mathsf{accesses}\ (k\,(\sigma\, l'))\ \sigma)$. Hence our hypothesis is literally the continuation's hypothesis. Since $\mathsf{runSeq}\ (\mathtt{read}\ l'\ k)\ \sigma = \mathsf{runSeq}\ (k\,(\sigma\, l'))\ \sigma$, the conclusion is literally the continuation's conclusion, and the induction hypothesis at $k\,(\sigma\, l')$ and σ finishes the case.

Case `write` $l'\ v\ k$. Now $W(\mathsf{accesses}\ (\mathtt{write}\ l'\ v\ k)\ \sigma) = [l'] +\!\!+ W(\mathsf{accesses}\ (k\,())\ \sigma[l' \mapsto v])$, and the hypothesis says l is in neither part. From the first part, $l \neq l'$. From the second, the induction hypothesis applies to $k\,()$ at the updated memory, giving $(\mathsf{runSeq}\ (k\,())\ \sigma[l' \mapsto v]).2\ l = \sigma[l' \mapsto v]\ l$. Since $l \neq l'$, equation (2) rewrites the right-hand side to $\sigma\ l$. Chaining these with $\mathsf{runSeq}\ (\mathtt{write}\ l'\ v\ k)\ \sigma = \mathsf{runSeq}\ (k\,())\ \sigma[l' \mapsto v]$ gives the claim. **This case is where** (2) **is spent; everything else is plumbing.**

Case `par` $cs\ k$. Writing $(vs, \tau) = \mathsf{runChilds}\ cs\ \sigma$, we have $W(\mathsf{accesses}\ (\mathtt{par}\ cs\ k)\ \sigma) = W(\mathsf{accChilds}\ cs\ \sigma) +\!\!+ W(\mathsf{accesses}\ (k\, vs)\ \tau)$, so l lies outside both. The children's conclusion gives $\tau\ l = \sigma\ l$; the continuation's induction hypothesis gives $(\mathsf{runSeq}\ (k\, vs)\ \tau).2\ l = \tau\ l$. Composing the two equalities gives the claim.

Children, empty spine. $\mathsf{runChilds}\ [\,]\ \sigma = ((), \sigma)$, so the claim is $\sigma\ l = \sigma\ l$.

Children, the 2*-to-*N *step.* From $\mathsf{accChilds}\ (p :: ps)\ \sigma = \mathsf{accesses}\, p\, \sigma \cup \mathsf{accChilds}\, ps\ (\mathsf{runSeq}\, p\, \sigma).2$ we get that l is outside the head's writes and outside the tail's writes. By the head's induction hypothesis, $(\mathsf{runSeq}\ p\ \sigma).2\ l = \sigma\ l$: the head left l alone, so the tail starts from a memory already agreeing with σ at l. By the tail's induction hypothesis at that memory, the tail leaves l alone too. Composing gives the claim. □

The moral. Lemma 32 is equation (2) lifted from one cell-write to an arbitrary program tree. Each case either does not touch memory or touches it once and invokes (2).

3.8.2 Locality

Definition 33. $\mathsf{FDisjoint}\ F\ f \coloneqq \big(F \cap R(f) = \emptyset\big) \wedge \big(F \cap W(f) = \emptyset\big)$.

Reads and writes are required disjoint from F *separately* rather than as one condition on their concatenation, because $\cup$ puts all the reads together and then all the writes, and a combined condition does not project through a nested union.

Lemma 34 (Locality)**.** *Suppose*

$$\mathsf{FDisjoint}\ F\ (\mathsf{accesses}\ p\ \sigma) \qquad \textit{(p does not touch F)} \tag{3}$$

$$\sigma\ l = \sigma'\ l \textit{ for every } l \notin F \qquad \textit{(the memories agree outside F)} \tag{4}$$

Then all three of the following hold:

$$\textit{(i)}\ \mathsf{accesses}\ p\ \sigma = \mathsf{accesses}\ p\ \sigma', \qquad \textit{(ii)}\ (\mathsf{runSeq}\ p\ \sigma).1 = (\mathsf{runSeq}\ p\ \sigma').1,$$

$$\textit{(iii)}\ (\mathsf{runSeq}\ p\ \sigma).2\ l = (\mathsf{runSeq}\ p\ \sigma').2\ l \textit{ for every } l \notin F,$$

and likewise at the children level.

Scratch work. One might hope to prove (ii) alone, since the answer is what we care about. It cannot be done. To get through a `read` node we must know the continuation is the *same program* on both sides, which needs the read to return the same value, which needs $l' \notin F$ — fine. But to get through the node *after* it we must know the two memories *still* agree outside F, which is (iii); and to apply hypothesis (3) to that continuation we must know its footprint matches, which is (i). Each conclusion is the fuel for the others. So we carry all three.

Proof. By mutual induction on p, carrying (i), (ii) and (iii) simultaneously.

Case `pure` a. Both footprints are empty, giving (i). Both runs return a, giving (ii). The final memories *are* σ and σ', which agree outside F by (4), giving (iii).

Case `read` $l'\ k$. The address l' is a read of this program, and hypothesis (3) says $F \cap R(\cdot) = \emptyset$, so

$$l' \notin F, \qquad \text{hence by (4)} \qquad \sigma\ l' = \sigma'\ l' =: v.$$

This is the heart of the lemma: the read returns the same value on both sides, so the continuation is literally the same program $k\,v$ in both runs and the two executions cannot diverge. Hypothesis (3) restricted to the second summand of the union gives $\mathsf{FDisjoint}\ F\ (\mathsf{accesses}\ (k\,v)\ \sigma)$, so the induction hypothesis applies to $k\,v$ at σ and σ'. Conclusions (ii) and (iii) transfer unchanged, because $\mathsf{runSeq}\ (\mathtt{read}\ l'\ k)\ \sigma = \mathsf{runSeq}\ (k\,v)\ \sigma$ and likewise for σ'; and (i) holds because both footprints are $([l'], [\,])$ unioned with the same thing.

Case `write` $l'\ v\ k$. Hypothesis (3), write half, gives $l' \notin F$. The continuation runs at the updated memories $\sigma[l' \mapsto v]$ and $\sigma'[l' \mapsto v]$, and we must check these still agree outside F — it is not automatic. Take any $m \notin F$. If $m = l'$, both memories give v by (1). If $m \neq l'$, both reduce to $\sigma\ m$ and $\sigma'\ m$ by (2), which agree by (4). So (4) is re-established at the updated memories, and (3) holds for the continuation. Apply the induction hypothesis; (i), (ii), (iii) follow as before.

Case `par` $cs\ k$. Write $(vs, \tau) = \mathsf{runChilds}\ cs\ \sigma$ and $(vs', \tau') = \mathsf{runChilds}\ cs\ \sigma'$. Hypothesis (3) applied to the union $\mathsf{accChilds}\ cs\ \sigma \cup \mathsf{accesses}\ (k\,vs)\ \tau$ gives $\mathsf{FDisjoint}\ F$ of both summands. The children's statement then gives $\mathsf{accChilds}\ cs\ \sigma = \mathsf{accChilds}\ cs\ \sigma'$, $vs = vs'$, and agreement of τ with τ' outside F. Because $vs = vs'$, the continuation is the *same program* $k\,vs$ on both sides; its footprint is disjoint from F and the two memories it starts from agree outside F, so the induction hypothesis applies and delivers (i), (ii), (iii).

Children, empty spine. Footprints are empty; answers are both $()$; memories are σ and σ', agreeing outside F by (4).

Children, the 2*-to-N step.* From $\mathsf{accChilds}\ (p :: ps)\ \sigma = \mathsf{accesses}\ p\ \sigma \cup \mathsf{accChilds}\ ps\ (\mathsf{runSeq}\ p\ \sigma).2$ and (3), both summands are disjoint from F. Apply the induction hypothesis to the head: same footprint, same answer, and the two post-memories agree outside F. Those post-memories are exactly where the tail starts, so (4) is re-established for the tail, and (3) for the tail is the second summand. Apply the induction hypothesis to the tail and compose. □

The moral. Locality says a program's behaviour depends only on the cells in its own footprint. It is the formal content of "what you do not look at cannot affect you", and it is the workhorse: Lemmas 35 and 41 are both instances of it, and Lemma 57 calls it twice.

3.8.3 Isolation

Now we cash in, and close the gap of Remark 16.

Lemma 35 (Isolation)**.** *If* $\mathsf{Indep}\ (\mathsf{accesses}\ p\ \sigma)\ (\mathsf{accesses}\ q\ (\mathsf{runSeq}\ p\ \sigma).2)$ *then*

$$\mathsf{accesses}\ q\ (\mathsf{runSeq}\ p\ \sigma).2 = \mathsf{accesses}\ q\ \sigma \qquad \textit{and} \qquad (\mathsf{runSeq}\ q\ (\mathsf{runSeq}\ p\ \sigma).2).1 = (\mathsf{runSeq}\ q\ \sigma).1.$$

In words: q ***cannot tell whether*** p ***has already run.***

Scratch work. We want Lemma 34 with the two memories being σ and $(\mathsf{runSeq}\ p\ \sigma).2$. Lemma 34 needs a set F such that q does not touch F and the two memories agree outside F. Which set could possibly work? The only cells where the two memories *can* differ are the cells p wrote. So take F to be exactly that set and check the two hypotheses against it. That is the entire proof.

Proof. Put $F \coloneqq W(\mathsf{accesses}\ p\ \sigma)$, and write $f_1 \coloneqq \mathsf{accesses}\ p\ \sigma$, $f_2 \coloneqq \mathsf{accesses}\ q\ (\mathsf{runSeq}\ p\ \sigma).2$.

Hypothesis (3)*, that* q *does not touch* F*.* Unfolding $\mathsf{FDisjoint}\ F\ f_2$ by Definition 33, we need two things:

$$F \cap R(f_2) = \emptyset \ \text{ i.e. } \ W(f_1) \cap R(f_2) = \emptyset, \qquad F \cap W(f_2) = \emptyset \ \text{ i.e. } \ W(f_1) \cap W(f_2) = \emptyset.$$

These are clauses (C2) and (C1) of Definition 17, verbatim. Nothing to do.

Hypothesis (4)*, that the two memories agree outside* F*.* We need $(\mathsf{runSeq}\ p\ \sigma).2\ l = \sigma\ l$ for every $l \notin F$. Since F is precisely $W(\mathsf{accesses}\ p\ \sigma)$, this says "$p$ leaves alone every cell it does not write" — which is Lemma 32, exactly.

Both hypotheses hold, so Lemma 34 applies to the program q with the two memories $(\mathsf{runSeq}\ p\ \sigma).2$ and σ. Its conclusions (i) and (ii) are the two claims. □

Remark 36. We used clauses (C1) and (C2) and never touched (C3). Clause (C3), namely $W(f_2) \cap R(f_1) = \emptyset$, is what the *opposite* order of the pair needs — it is q not disturbing p. Definition 17 is symmetric because both orders come up.

Example 37 (The gap, closed, on numbers). With p, q, σ_0, σ_1 as in Example 11:

$$\mathsf{accesses}\ q\ \sigma_1 = ([7],[8]) \qquad \text{and} \qquad \mathsf{accesses}\ q\ \sigma_0 = ([7],[8]).$$

Equal — and the reason is that when q reads cell 7 it gets $\sigma_1\,7$ in the first case and $\sigma_0\,7$ in the second, and $\sigma_1\,7 = \sigma_0\,7$ precisely because $7 \notin W(\mathsf{accesses}\ p\ \sigma_0) = \{5\}$, which is clause (C2).

Now substitute the broken q', which reads cell 5. The footprints still match, $([5],[8])$ either way, but the *answers* do not: q' writes 1 into cell 8 when measured after p and 0 when measured before. Lemma 35's conclusion fails — and it must, because its hypothesis fails: $W(f_1) \cap R(f_2) = \{5\} \neq \emptyset$.

The moral. An independent branch really is computed in isolation: its footprint and its result do not depend on whether the other branch has run. This is the content that separation logic expresses as a splitting $\sigma_1 * \sigma_2$ of the heap — obtained here without splitting anything, and without a condition on shared reads.

3.8.4 No race, and locality of the obligation

Lemma 38 (Ready actions lie inside footprints)**.** *Every label in* nextAccs p *has its address in* W(accesses $p\ \sigma$) *if it is a write and in* R(accesses $p\ \sigma$) *if it is a read; likewise for* nextAccsChilds *against* accChilds*.*

Proof. Mutual induction on p. For `pure`, nextAccs is empty and there is nothing to check. For `read` $l\ k$, the single ready label is `read` l and Definition 13 puts l in the reads; `write` is symmetric. For a fork, the ready labels are exactly the children's labels flattened, and accesses contains accChilds $cs\ \sigma$ as a summand, which by the children's induction hypothesis contains every child's ready address — and a union only adds. The 2-to-N step: a ready label belongs to the head or to one of the rest, and either way the union keeps it. □

Lemma 39 (Independence excludes a race)**.** *If* Indep (accesses $P\ \sigma$) (accesses $Q\ \sigma$) *then* racing (nextAccs P) (nextAccs Q) *is false. Consequently, if* IndepAll (childFootprints $cs\ \sigma$) *then* racingAll (nextAccsChilds cs) *is false.*

Scratch work. The claim is negative — "no race" — which is the standard signal for proof by contradiction. So suppose there *is* a conflicting pair and find which Bernstein clause it violates.

Proof. Suppose racing (nextAccs P) (nextAccs Q) is true. By Definition 24 there are labels $a \in$ nextAccs P and $b \in$ nextAccs Q that conflict, so by Definition 23 they name one address l and at least one is a write. Three cases, and by Lemma 38 each puts l in a particular component of each side's footprint.

- *Both writes.* Then $l \in W(f_1) \cap W(f_2)$, contradicting (C1).
- *a a write, b a read.* Then $l \in W(f_1) \cap R(f_2)$, contradicting (C2).
- *a a read, b a write.* Then $l \in W(f_2) \cap R(f_1)$, contradicting (C3).

Every case contradicts a hypothesis, so no such pair exists.

For the N-ary consequence, induct on the children. The head cannot race any later child: IndepAll's first conjunct gives independence of the head's footprint from each later child's, hence — by a union argument — from their combined footprint, to which the first part applies. The later children cannot race each other by the induction hypothesis. This is the 2-to-N step for the race check. □

Remark 40. Lemma 35 used two of Bernstein's clauses; Lemma 39 uses all three. Between them the definition is used exactly, with nothing left over — which is one reason to believe Definition 17 has the right number of clauses.

Lemma 41 (Locality of the obligation)**.** WellFormed *and* childFootprints *are invariant under changing the memory off a footprint-disjoint set* F*.*

Proof. Mutual induction on the program, using Lemma 34 at each node to transport the footprints the obligation is stated about. There is no new idea here; the lemma exists because Lemma 57 needs to move a hypothesis, not just a value, from one memory to another. □

4 The surface language

Everything so far has been about `ParM`, which is a *type* of programs rather than a language anyone writes. Its effect nodes carry Lean functions as continuations, and two consequences follow for anything built on top. A `ParM` term cannot be printed past its first `read`, because what comes after a read *is* a function of the value read; and its footprint has to be obtained by

simulating it against a memory (Definition 13), because the addresses a continuation will touch are not visible in the term. Both are tolerable when you assemble programs by hand out of combinators. Neither survives contact with a compiler, which delivers instructions rather than closures.

`ParProg` is the same programs with their computations as *data*. It is a first-order inductive type — `load` and `store` for memory, `set` for register arithmetic, `seq`, `par` over a list of threads, `label` for a named region, `ifElse`, and `loadAt`/`storeAt` for an address computed at run time — together with two interpretations into `ParM`: `execPar`, which sends `par` to the `ParM` `par` constructor, and `execSeq`, which sends it to the corresponding chain of binds.

No new semantics is introduced and no axiom is added. Every theorem in this section ends in a statement about `ParM` and is discharged by §5. What the layer contributes is that the three things you actually want — printing the program, knowing its footprint, and discharging its obligation — become *computations* instead of proofs.

Two structural facts about the language are worth stating, because the independence obligation depends on them. The only parallel form is a block of threads, and it is fork–join: threads are spawned together and joined together, with no construct letting one wait on, signal, or lock against another. And registers are thread-private — a `par` node hands each thread a copy of the register file and discards what the threads leave in it. That is why a thread cannot return a value (it writes one), and it is why the obligation is about memory alone.

4.1 Memory is declared, not allocated

An address is a number, and a program of any size needs those numbers to come from somewhere other than the author's head. A *layout* is where they come from. A declaration carries a name, a shape and initial contents and *no address*; the allocator places each buffer where the previous one ended; and a subscript is resolved against the shape, row-major, so $\mathtt{A}[i, j]$ is the literal $base_A + i \cdot c + j$ — an ordinary constant by the time §4.2 folds it. Nothing about `ParM` or §5 knows this layer exists.

What it buys is one hypothesis, discharged once instead of once per pair of arrays.

Proposition 42 (Placement is disjoint)**.** *Buffers placed consecutively from any start address are pairwise non-overlapping: of any two, one ends no later than the other begins.*

A declaration can also end. `free` A does two things and nothing else: the name leaves scope, so a later mention of A fails to compile; and when A is the most recent buffer placed, its cells go back to the cursor, so the next declaration is placed on them. Freeing out of order leaves the hole, since the cursor only moves back from the top. Nothing is zeroed and no location goes away — a buffer declared on a freed range begins holding what the previous occupant left. This is an arena, not a heap, and the fact to want about it is not Proposition 42, which reuse falsifies on purpose, but this one:

Proposition 43 (Live buffers are disjoint)**.** *In the layout produced by any list of declarations and frees, two buffers that are live at the same time share no location.*

Proof. Carry two invariants along the declaration list: every live buffer's cells lie below the cursor, and no cell belongs to two live buffers. A declaration places a buffer at the cursor and advances it, so the new buffer lies below the new cursor and, by the first invariant, overlaps no existing live buffer. A free is where the two invariants meet: lowering the cursor could strand a live buffer above it only if that buffer lay inside the range just freed, which the second invariant has already excluded. Disjointness is stated over *cells*, which makes a zero-size buffer vacuously disjoint from everything. □

Why Definition 1 is unaffected. Both forms are *erased*. Neither emits a `ParProg` node, so `buf` $A[2]$; $A[0] \coloneqq 1$; `free` A compiles to $[0] \coloneqq 1$ on the nose. The placement happens in the compiler, before there is a program to run: State is still $\mathsf{Loc} \to \mathsf{Val}$, still total, and still has no operation that creates or destroys a location — which is why "no allocation" remains the right description of the semantics while the language in front of it declares and reclaims memory. What a set of declarations decides is which addresses the program names and what the initial memory is, and every theorem from §5 on is parametric in the memory. So a different layout changes the numbers in an example and never an obligation.

Why there is no run-time version. Admitting `alloc` and `free` as *statements* would cost Theorem 62 twice over. An allocator has to choose an address; two schedules choose differently, so the final memories differ and an equality of memories is false for reasons that have nothing to do with races. A run-time `free` is worse: Bernstein's conditions are about reads and writes of locations, so two threads freeing one block, or one freeing while another reads it, is an error that three empty intersections cannot see, and Indep would have to be re-derived rather than reused. Compile-time placement avoids both by leaving no trace in the program.

4.2 The footprint is computed, not proved

footprintOf reads a footprint off the syntax by a fold. The two interesting clauses are the ones where an address is not a literal:

```
def footprintOf : ParProg -> Footprint
  | .load _ l      => ([l], [])        | .store l _     => ([], [l])
  | .seq a b       => union (footprintOf a) (footprintOf b)
  | .par ss        => footprintsOf ss  | .label _ s     => footprintOf s
  | .loadAt _ a _  => (a.cells, [])    -- index unknown: bound by the whole array
  | .storeAt a _ _ => ([], a.cells)
  | .ifElse _ t e  => union (footprintOf t) (footprintOf e)
```

It costs nothing to evaluate, and it is related to the real footprint by one proposition.

Proposition 44 (The syntactic footprint bounds the real one)**.** *For every s : `ParProg`, register file ρ and memory σ,*

$$\mathsf{accesses}\ (\mathsf{execPar}\ s\ \rho)\ \sigma\ \sqsubseteq\ \mathsf{footprintOf}\ s,$$

where $f \sqsubseteq g$ means containment of reads in reads and writes in writes. If moreover s contains no `loadAt`, `storeAt` or `ifElse`, the containment is an equality.

Proof. Induction on s using the mutual eliminator of §2.3. The literal cases are equalities by Definition 13. For `ifElse` the real footprint is that of whichever arm runs, and each arm is contained in the union of the two. For `loadAt` and `storeAt` the index wraps on the array's length by construction, so the real access lies inside the array's cells, which is the stated bound. □

Containment is all the obligation needs, and that is a fact about Bernstein rather than a convenience.

Lemma 45 (Independence is antitone)**.** *If $f' \sqsubseteq f$ and $g' \sqsubseteq g$ and* Indep $f\ g$*, then* Indep $f'\ g'$*.*

Proof. All three clauses of Definition 17 assert that an intersection is empty, and shrinking either operand of an intersection cannot make it non-empty. Each clause transfers directly. □

Remark 46. Lemma 45 is why over-approximating a footprint is sound: widening a read set can never manufacture a conflict, because there is *no clause* on $R(p) \cap R(q)$ to violate. Under $*$-disjointness the same move would be unsound, since there enlarging a footprint enlarges the claimed ownership. The missing fourth clause of Definition 17 thus pays for itself a second time.

4.3 The obligation is decided

wfB is Definition 21 read off the syntax: Bernstein at every `par` node, between every pair of its threads.

```
def wfB : ParProg -> Bool
  | .load _ _ | .store _ _ | .set _ _ => true
  | .seq a b                         => wfB a && wfB b
  | .par ss       => indepAllB (threadFootprints ss) && wfAllB ss
  | .label _ s    => wfB s
  | .ifElse _ t e => wfB t && wfB e    -- either branch may run
```

Theorem 47 (Equivalence and cost, for the language)**.** *Let* s : *`ParProg` with* wfB s = `true`*. Then for every register file* ρ *and memory* σ*,*

$$\mathsf{execPar}\ s\ \rho\ \|\equiv \mathsf{execSeq}\ s\ \rho \textit{ from } \sigma, \qquad \mathsf{time}\ (\mathsf{execPar}\ s\ \rho)\ \sigma\ \leq\ \mathsf{time}\ (\mathsf{execSeq}\ s\ \rho)\ \sigma.$$

Proof. At each `par` node, wfB checks independence of the threads' *syntactic* footprints — read off the program text, with no memory in sight. But WellFormed is a statement about *real* footprints, at every memory. Two steps bridge them.

First, Proposition 44: at any memory, a thread's real footprint is contained in its syntactic one, componentwise. Second, Lemma 45: independence is preserved by shrinking either argument. So syntactic independence implies real independence, and it does so *at every memory at once*, because the syntactic footprint does not depend on the memory. That universal quantifier is exactly what Definition 21 asks for, so WellFormed (execPar s ρ) σ holds for all ρ and σ from one Boolean. Note the direction: over-approximating can only make the check *harder* to pass, never unsound.

The two interpretations have the same canonical run, by induction on s. Then Theorem 62 gives the equality and §6 the inequality, from the same hypothesis. □

The hypothesis of Theorem 47 is a closed Boolean expression, so for any particular program it is discharged by `decide` — evaluation inside the kernel, with no proof term written by hand. The same Boolean licenses running the program on real hardware: an interpreter that spawns a thread at every `par` node agrees with runSeq whenever wfB is true, so a benchmark of the threaded run against the single-threaded one measures speed and nothing else — the answers are equal by theorem, not by observation. This is the one place real threads appear, and they appear as a compilation target, never as the thing the theorem is about (§2.4).

4.4 Pipelines and kernels

In practice a parallel program is not one fork but a chain of phases, several of them sequential. Because wfB on `seq` and on `label` is definitional conjunction, a chain's obligation is the conjunction of its phases'.

Proposition 48 (Pipelines compose)**.** *If* wfB s = `true` *for every phase* s *of a list* ps*, then the parallel and sequential readings of the whole block are outcome-equivalent from every memory. A phase containing no `par` node satisfies the premise outright.*

The second half is what makes the first useful. Sequential code carries *no* obligation, and not because we ignore it: there is no parallelism in it for the sequential reading to differ from. So a pipeline costs exactly one obligation per parallel phase, however much ordinary code surrounds them.

A phase is rarely written thread by thread. parFor n b is the `par` node whose threads are $b\,0, \ldots, b\,(n-1)$. For a concrete n its obligation is still a `decide`, but `decide` reduces the generated list, so the cost grows with the program's *size* rather than the difficulty of the argument

— and it grows fast. The check is pairwise over threads, so a kernel with n^2 threads each touching $\Theta(n)$ cells costs $\Theta(n^5)$: on a surface matrix multiply, wfB at $n = 6, 8, 10, 12$ takes 1.5, 6.3, 19.3 and 49.3 seconds, fitting an exponent of 5.1.

The way out is to prove the loop once, for every n — a lemma rather than an evaluation.

Proposition 49 (A loop's obligation reduces to its bodies)**.** *Let* $rd, wr : \mathbb{N} \to \mathsf{List\ Loc}$ *with* footprintOf $(b\,i) = (rd\,i, wr\,i)$ *for every* i*. If every body is well-formed, and for all* $i \neq j$ *below* n *the sets* $wr\,i$ *and* $wr\,j$ *are disjoint and* $wr\,i$ *and* $rd\,j$ *are disjoint, then* wfB (parFor n b) = `true`*.*

The premises are arithmetic about index expressions — what the author actually knows about the decomposition, and nothing more. On the same matrix multiply the whole obligation, at every n and from two range inequalities about the three blocks' base addresses, elaborates in under a fifth of a second, against 19.3 seconds for the single size $n = 10$.

Each shape worth reusing is then packaged as a *kernel*: a function returning a `ParProg`, with its obligation proved once by Proposition 49 and stated for every size. Eleven are supplied; the third column is the point.

Kernel	**Thread i (or (i, j)) does**	**Sharing**
mapK	$\mathrm{dst}[i] := f(\mathrm{src}[i])$	none
zipK	$\mathrm{out}[i] := f(a[i], b[i])$	none
broadcastK	reads all of *srcs*, writes $\mathrm{dst}[i]$	total, on reads
stencilK	$\mathrm{out}[i] := \sum_{k<w} c_k \cdot \mathrm{inp}[i{+}k]$	overlapping windows
conv1dK	as above with the weights in memory	windows and weights
transposeK	$\mathrm{out}[j, i] := \mathrm{inp}[i, j]$	none; both sides permute
biasAddK	$\mathrm{out}[i, j] := \mathrm{inp}[i, j] + \mathrm{bias}[j]$	a column, across rows
denseK	$\mathrm{out}[i] := \sum_j W[i, j]\, x[j] + b[i]$	all of x
outerUpdateK	$W[i, j] \mathrel{-}= \mathrm{lr} \cdot e[i]\, x[j]$, in place	$e[i]$, $x[j]$
reduceK	folds K cells into $\mathrm{out}(i)$	whatever *locs* says
gridK	folds K cells into $\mathrm{out}(i, j)$	whatever *locs* says

Every row but the first two has threads whose read sets overlap — in denseK and broadcastK, completely — while their write sets stay disjoint. That is Bernstein's case exactly, and it is the case $*$-disjointness rejects, so the table is also a list of kernels a $*$-based rule cannot admit without fractional permissions. outerUpdateK is the sharpest: each thread reads *and writes* $W[i, j]$ in place while sharing $e[i]$ with its row and $x[j]$ with its column, and it is well-formed because ownership here is per cell rather than per region.

Remark 50 (A kernel is a builder, not a certificate). This is what keeps the convenience sound. The lemma attached to a kernel is an implication whose antecedent is about the instantiation — for mapK, that $\mathrm{dst} + i \neq \mathrm{src} + j$ for all $i, j < n$ — and wfB is computed from footprintOf of the program that was actually produced, with no knowledge that it came from a kernel at all. Instantiating mapK with overlapping source and destination therefore yields a program whose wfB is `false`, and the lemma that would have proved it well-formed cannot be applied, because its side condition is false. The check sees the instantiation, never the promise.

4.5 Computed addresses and branches

Two constructs break the correspondence between syntax and footprint, and both are needed for real programs. `ifElse` branches on a register, and `loadAt`/`storeAt` take an address computed at run time — the $A[B[i]]$ of a scatter or a gather, where no static analysis of the index is possible in principle.

For a branch, footprintOf returns the union of the two arms, because which arm runs is not known until the memory is. For a computed address it returns the whole of the array indexed: an array reference carries a base and a length, the index wraps on that length, so the access

is confined to the array by construction and the array is the bound. A bounded loop is then derived rather than primitive — it is n nested `ifElse`s — so it inherits the same treatment and *terminates*, which is what keeps it expressible at all, `ParM` being inductive (§2.2).

What this costs is precision, never soundness, and the tiers are easy to tell apart. Proposition 44 is a containment for *every* program, so Theorem 47 holds whether or not the addresses are literal. What fails for a computed address is the *equality*, and with it the ability to tell that two threads indexing the same array do not collide — so two threads scattering into A are rejected even when their indices never meet. When that matters, an exact check is available per memory: a scan of the actual run is both sound and complete for WellFormed, and when it fails it produces the schedule that goes wrong.

4.6 Proposing a parallelisation

The analysis can also run in the other direction: given a sequential program, where *could* the forks go? A `ParProg` carries its own footprint, so "can these two run together?" is a Boolean, and the proposal is a short program — flatten the chain, group a maximal pairwise-independent run, fork it, repeat. The architecture is the standard one for untrusted search: the analysis is **untrusted**, and wfB is the proved check that accepts or rejects what it produces.

That architecture has a gap, and it is instructive, because the gap is exactly what an independence criterion cannot see.

What wfB cannot check. A `par` node hands every thread a copy of the *pre-fork* register file and returns that same file to the continuation. Register writes do not survive a join. They do not survive it in the *sequential* reading either, so the two readings agree, no race occurs, and wfB returns `true` — while the answer changes. On

$$[10]{:=}1;\quad [11]{:=}2;\quad \%0{:=}[10];\ \%1{:=}[11];\ [12]{:=}\mathtt{R0}{+}\mathtt{R1};\quad \%2{:=}[11];\ [13]{:=}\mathtt{R2}{*}2$$

a scheduler working statement by statement puts the two loads in one stage and the store that consumes them in the next, where the fork has already discarded them. The original leaves $M[12] = 3$ and $M[13] = 4$; the proposal leaves 0 and 0, with wfB = `true`.

The lesson is not that the check is too weak. It is that memory dependences and register dependences need *opposite* treatment: a memory dependence is discharged by sequencing two statements, and a register dependence is *destroyed* by sequencing them across a fork.

The unit of scheduling is a segment. A boundary may be placed only where no register is *live* across it, so the schedulable atoms are register-closed segments rather than statements: a load and the store consuming its register are one segment and are never separated. Two refinements are each forced by a program.

Liveness must kill. A reused register — $\%0{:=}[\mathtt{i}];\ \mathtt{A}[\mathtt{i}]{:=}\mathtt{R0}$, repeated — is written before every use, so it is never live across a boundary and the copies fork. Taking "written somewhere earlier" instead of "live" would make %0 appear live from the first statement to the last, and no boundary would ever be legal.

The kill must be a must-write. A register write set is a *may*-write — a conditional's is the union of its branches, since either may run. Liveness that kills on it is unsound:

$$\%0{:=}5;\quad \mathtt{if\ (R1)}\ \{\%0{:=}1\}\ \mathtt{else}\ \{[7]{:=}9\};\quad [8]{:=}\mathtt{R0}$$

Only the `then` branch assigns %0, so killing on the union declares it dead, offers a boundary after the first statement, and $M[8]$ reads 0 instead of 5. The kill set of a conditional is the *intersection* of its branches'; that of a `par` node is empty, because a fork writes no register its parent can see.

Registers constrain nothing else. Whether two segments may be *siblings* is memory-Bernstein alone: threads get a private file, so siblings may clobber the same register freely, and with a reused %0 they routinely do. The register condition is a property of the *boundary*, not of a pair.

The theorem. What is then true of the rewrite is not an independence statement but an equality of outputs, and it is proved rather than checked.

Theorem 51 (Forking a chain preserves its memory)**.** *Let g be a list of segments in which no register written by one is read by a later one. Then for every register file ρ and memory σ,*

$$\big(\mathsf{runSeq}\ (\mathsf{execPar}\ (\mathit{par}\ g)\ \rho)\ \sigma\big)_2 \;=\; \big(\mathsf{runSeq}\ (\mathsf{execSeq}\ (\mathsf{block}\ g)\ \rho)\ \sigma\big)_2.$$

Proof. By induction on g, carrying the set of registers written so far. Two frame lemmas do the work. First, a statement changes only the registers its register write set names. Second, if two register files agree on a set R covering what a statement reads, the statement leaves the same memory under both and the files still agree on R afterwards. The second is stated with R as a *parameter* rather than with the statement's own read set, because a conditional's write set names registers only one branch assigns — so "the files agree on the writes" is false for `ifElse`, and the parameterised form is what carries through a sequence. □

The conclusion is about the memory, not the pair: the register file after a fork is the pre-fork one by construction, so the memory is what the two readings can share, and it is the program's output. Composed with Theorem 47 at the proposal, it says the forked program computes the original's memory under every schedule.

Fork–join is series-parallel; dependence DAGs are not. This is the standing limitation of the whole approach, and no better scheduler fixes it. Every program expressible here is a series-parallel DAG. The smallest obstruction is the **N** — the program above: $i_1 \to i_3$, $i_2 \to i_3$, $i_2 \to i_4$, with i_1 and i_4 unrelated. No fork–join term can start i_4 before i_1 finishes, because i_4 is either parallel to i_1, and then cannot wait for i_2, or after it. With i_1 and i_4 made long, the span this costs is measurable in the cost semantics of §6:

Shape	**Span**
sequential	17
$\texttt{par}[i_1, i_2];\ \texttt{par}[i_3, i_4]$ — stage-wise, what the analysis emits	13
$\texttt{par}[i_1,\ (i_2; i_4)];\ i_3$ — nested	11
$i_2;\ \texttt{par}[i_4,\ (i_1; i_3)]$ — nested	**10**
the dependence DAG's critical path	**9**

Nesting is worth $13 \to 10$, and the remaining $10 > 9$ is irreducible. Recognising series-parallel digraphs is linear time, but no algorithm for embedding a general DAG in one is known to bound the critical-path increase by a constant factor. The N is also why no *online* scheduler can be optimal: having forked $i_1 \parallel i_2$ and joined for i_3, it has killed i_2's thread before i_4 arrives.

5 Canonical outcomes, and the equivalence

Everything so far has been definitions and locality. This section proves the theorem they were built for, and then reads the equivalence off it in ten lines.

5.1 The theorem, and how to attack it

Theorem 52 (Canonical outcomes)**.** *For every program p, memory σ, answer r and memory σ',*

$$\mathsf{WellFormed}\ p\ \sigma \ \longrightarrow\ \Big(\mathsf{Reaches}\ \langle p,\sigma\rangle\ \langle \mathit{pure}\ r,\sigma'\rangle\ \longleftrightarrow\ (r,\sigma') = \mathsf{runSeq}\ p\ \sigma\Big).$$

Read it slowly. It says: **for a well-formed program, the set of states it can possibly finish in has exactly one element, and that element is** $\mathsf{runSeq}\ p\ \sigma$**.** This is confluence and normalisation in one biconditional, and both halves do real work.

Left to right is safety: no schedule terminates anywhere other than the canonical outcome. *Right to left is liveness*: the canonical outcome really is reached, so the program has not been proved deterministic by having no terminating executions at all. A soundness theorem in the usual sense gives only the first.

Scratch work. How does one even attack this? Two natural ideas both fail, and seeing why is the fastest route to the one that works.

Attempt 1: induct on the number of steps, comparing the two programs directly. We would want "after n steps the parallel program is in the same state as the sequential program after m". This is false. After one step the parallel program may have performed q's read while the sequential program has definitely performed p's write; the two machines are in genuinely different states, and there is no step-by-step correspondence to be had.

Attempt 2: enumerate the schedules. For two children with n and m steps there are $\binom{n+m}{n}$ interleavings, and children nest arbitrarily. There is no finite case analysis here.

Attempt 3, which works. Forget comparing the two programs to each other. Instead find a quantity that (a) can be computed from any configuration, (b) *never changes* as the machine steps, and (c) at the end of a run equals the answer. If such a quantity exists then every terminating path, however wild, must end at the same answer — because the quantity was fixed at the start and the end merely reads it off. This is a conservation law: whatever route a ball rolls down a hill, its energy at the bottom is the same, so you do not need to know the route.

And we already have the candidate: runSeq. It is computable from any configuration $\langle P,\sigma\rangle$ — just apply the function — and at a finished configuration $\langle \mathtt{pure}\ r,\sigma'\rangle$ it gives (r,σ') immediately by Definition 9's first line.

So the plan, writing $\Phi(P,\sigma) \coloneqq \mathsf{runSeq}\ P\ \sigma$, is:

1. Φ never changes when the machine takes a step. (Lemma 57, *Conservation*)
2. $\mathsf{WellFormed}$ never stops holding when it takes a step, so step 1 applies again. (Lemma 60, *Preservation*)
3. Hence by induction Φ is constant along every path. (Lemma 61, *Chaining*)
4. At a finished configuration, Φ reads off the answer. (Definition 9)

That gives safety. Liveness needs a different idea entirely, and gets §5.7.

The difficulty, on two children. Before the machinery, here is the whole problem in one example. Take the fork $\mathtt{par}\ \Pi[\,p\ q\,]\ k$ with p and q as in Example 6, started at σ_0. The reference run runSeq is left-first: it runs p to completion, reaching $\sigma_1 = \sigma_0[5 \mapsto 1]$, and only then runs q — so the value q reads from cell 7 is $\sigma_1\,7$. A real schedule is free to step q first, and then the value q reads is $\sigma_0\,7$. **The two runs measure q at different memories, and nothing so far says they agree.**

Independence is what closes the gap. Here p writes $\{5\}$ while q reads $\{7\}$ and writes $\{8\}$, so Definition 17's three clauses hold, so σ_0 and σ_1 agree at every cell q looks at, and by Lemma 35 q computes the same value and the same footprint from either. The same is true of p when q has moved first. Every case of the proof below is this situation, at n children instead of two and at an arbitrary depth of nesting.

The induction. The proof is a mutual induction, and its children-level motive is the only design decision in the argument.

Definition 53 (A finished spine)**.** pureChilds vs is the spine of the same length and index in which every child is a finished leaf: at position i it holds `pure` (vs_i). It is the configuration a fork's children must reach before the join can fire, since allPure is defined exactly there.

Theorem 54 (The pair that is proved)**.** *The following are proved simultaneously by mutual induction:*

$$\begin{aligned} \mathit{motive}_1\ p &:= \forall \sigma\, r\, \sigma',\ \mathsf{WellFormed}\ p\ \sigma \to \big(\mathsf{Reaches}\ \langle p, \sigma\rangle\ \langle \texttt{pure}\ r, \sigma'\rangle \leftrightarrow (r, \sigma') = \mathsf{runSeq}\ p\ \sigma\big), \\ \mathit{motive}_2\ cs &:= \forall \sigma,\ \mathsf{wfChilds}\ cs\ \sigma \to \mathsf{IndepAll}\ (\mathsf{childFootprints}\ cs\ \sigma) \to \\ &\qquad \mathsf{ReachesChilds}\ cs\ \sigma\ \big(\mathsf{pureChilds}\ (\mathsf{runChilds}\ cs\ \sigma).1\big)\ (\mathsf{runChilds}\ cs\ \sigma).2. \end{aligned}$$

motive_2 says: *you can run every child to completion, left to right, by moves of* stepChilds. It is the N-ary form of "run p fully, then q", and it is mutual with motive_1 because running one child to completion *is* Theorem 52 one level down. Its `cons` case is the 2-to-N step.

The argument assembles four pieces. We give each, then the assembly.

5.2 Piece 1: a run out of a fork is in one of three states

Lemma 55 (Join-point dichotomy)**.** *If* $\mathsf{Reaches}\ \langle \texttt{par}\ cs\ k, \sigma\rangle\ C$ *then one of:*

1. $C = \langle \texttt{par}\ cs'\ k, \sigma''\rangle$ *for some* cs', σ'' *— still at the fork, children mid-flight;*
2. *there are* vs, σ'' *with* $\mathsf{Reaches}\ \langle \texttt{par}\ cs\ k, \sigma\rangle\ \langle k\, vs, \sigma''\rangle$ *and* $\mathsf{Reaches}\ \langle k\, vs, \sigma''\rangle\ C$ *— the join has happened;*
3. $C = \mathbf{wrong}$.

Proof. Induction on the closure (Definition 29). The base case is (1), with $cs' = cs$. For the inductive step, a configuration of form (1) can step only as Definition 26's trichotomy allows: to **wrong**, giving (3); to another `par` configuration via stepChilds, staying in (1); or — when $\mathsf{allPure}\ cs'$ is defined — past the join into k, giving (2). Cases (2) and (3) are closed under stepping, since (2) only extends its second run and **wrong** has no successors. □

Lemma 55 pins down *that* a join happens and that nothing else can. Which results and memory it lands on is Piece 3's job. The N-ary shape costs nothing here, because "a child moved" is a single case no matter how many children there are.

5.3 Piece 2: children's runs lift to the fork

Lemma 56 (Lifting)**.** *1. If* $\mathsf{Reaches}\ \langle p, \sigma\rangle\ C$ *then either* $C = \mathbf{wrong}$*, or* $C = \langle p', \sigma'\rangle$ *with* $\mathsf{ReachesChilds}\ (p :: ps)\ \sigma\ (p' :: ps)\ \sigma'$.

2. *If* $\mathsf{ReachesChilds}\ ps\ \sigma\ ps'\ \sigma'$ *then* $\mathsf{ReachesChilds}\ (p :: ps)\ \sigma\ (p :: ps')\ \sigma'$.
3. *If* $\mathsf{ReachesChilds}\ cs\ \sigma\ cs'\ \sigma'$ *and no configuration reachable along that run races, then* $\mathsf{Reaches}\ \langle \texttt{par}\ cs\ k, \sigma\rangle\ \langle \texttt{par}\ cs'\ k, \sigma'\rangle$.

Proof. Parts (1) and (2) are inductions on the closure, injecting each move into the left or the right summand of stepChilds's successor list — which by Definition 26 is the head's moves appended to the tail's, so a move of the head lands in the first part and a move of the tail in the second.

Part (3) is an induction in which each children-level move must be shown to be a legal *fork* move. By Definition 26 the fork at each configuration could instead go to **wrong** or join. The first is excluded because the race test is false by hypothesis. The second is excluded because a child that can move means allPure is undefined, which is Proposition 27 read in the contrapositive. □

Together, (1) and (2) are the 2-to-N step for reachability: run the head to completion with the tail standing still, then run the tail from the memory the head left.

5.4 Piece 3: one step preserves the canonical outcome

This is where WellFormed gets used, and it is the main lemma of the development.

Note first what independence is *not* needed for. The race test of Definition 26 runs at every step, so a real step that does not crash is automatically non-racing at that instant, whatever the hypothesis. Independence is needed for something narrower: when a child moves before an earlier child has finished, its real starting memory is the joint memory, not the left-first reference point runSeq is stated against, and only independence makes replaying that step against the reference point give the same answer.

Lemma 57 (Conservation)**.** *If* WellFormed $P\ \sigma$ *and* $\langle P_3, \sigma_3\rangle \in \mathsf{stepC}\ \langle P, \sigma\rangle$*, then*

$$\mathsf{runSeq}\ P_3\ \sigma_3 = \mathsf{runSeq}\ P\ \sigma \tag{5}$$

$$\forall F,\ \mathsf{FDisjoint}\ F\ (\mathsf{accesses}\ P\ \sigma) \to \mathsf{FDisjoint}\ F\ (\mathsf{accesses}\ P_3\ \sigma_3) \tag{6}$$

and correspondingly at the children level, given IndepAll *of the child footprints.*

Conclusion (5) is the conservation law. Conclusion (6) says a step never lets the footprint reach into new territory; §5.4.1 explains why it must be carried alongside (5) rather than proved afterwards.

Proof. By mutual induction on P. Six cases.

Case `pure` a. By Definition 26, step (`pure` a) $\sigma = [\,]$, so there is no $\langle P_3, \sigma_3\rangle$ in it: the hypothesis is false and there is nothing to prove.

Case `read` $l\ k$. The successor list is the singleton $[\langle k\,(\sigma\,l),\ \sigma\rangle]$, so $P_3 = k\,(\sigma\,l)$ and $\sigma_3 = \sigma$.

For (5): by Definition 9, runSeq (`read` $l\ k$) σ = runSeq $(k\,(\sigma\,l))\ \sigma$, which is *literally* runSeq $P_3\ \sigma_3$. The two sides are the same term.

For (6): by Definition 13, accesses (`read` $l\ k$) $\sigma = ([l], [\,]) \cup$ accesses $(k\,(\sigma\,l))\ \sigma$. A set disjoint from the whole union is in particular disjoint from the right summand, which is accesses $P_3\ \sigma_3$.

Observe that no independence was used. A `read` step is deterministic, and runSeq's own defining equation *is* the claim.

Case `write` $l\ v\ k$. The successor list is $[\langle k\,(),\ \sigma[l \mapsto v]\rangle]$, and runSeq (`write` $l\ v\ k$) σ = runSeq $(k\,())\ \sigma[l \mapsto v]$ — again the same term. Conclusion (6) is the same union argument.

Case `par` $cs\ k$. Write $(vs_0, \tau_0) =$ runChilds $cs\ \sigma$, so that by Definition 9

$$\mathsf{runSeq}\ (\mathtt{par}\ cs\ k)\ \sigma = \mathsf{runSeq}\ (k\,vs_0)\ \tau_0. \tag{7}$$

Unfolding WellFormed (`par` $cs\ k$) σ by Definition 21 gives the three facts we may use:

(W1) IndepAll (childFootprints $cs\ \sigma$), (W2) wfChilds $cs\ \sigma$, (W3) WellFormed $(k\,vs_0)\ \tau_0$.

Now split by the trichotomy of Definition 26.

(i) racingAll *is true.* The only successor is **wrong**, which is not of the form $\langle P_3, \sigma_3\rangle$, so the hypothesis cannot be satisfied. Nothing to prove.

(ii) The join fires, allPure $cs =$ some vs. The only successor is $\langle k\,vs,\ \sigma\rangle$. By (7) it suffices to show runSeq $(k\,vs)\ \sigma$ = runSeq $(k\,vs_0)\ \tau_0$, and for that it suffices to show runChilds $cs\ \sigma = (vs, \sigma)$ — i.e. that $vs_0 = vs$ and $\tau_0 = \sigma$. This is a small lemma of its own: *if every child is a finished leaf, running them changes nothing.* Its proof is an induction on cs: for the empty spine runChilds $[\,]\ \sigma = ((), \sigma)$; for `pure` $a :: ps$ we have runSeq (`pure` a) $\sigma = (a, \sigma)$, so the tail

starts at the same σ and the induction hypothesis closes it. Given that, (5) holds; and for (6), $\mathsf{accesses}\ (\mathtt{par}\ cs\ k)\ \sigma = \mathsf{accChilds}\ cs\ \sigma \cup \mathsf{accesses}\ (k\, vs_0)\ \tau_0$ whose right summand is now exactly $\mathsf{accesses}\ P_3\ \sigma_3$.

(iii) A child moves. Then $P_3 = \mathtt{par}\ cs_3\ k$ with σ_3 coming from $\mathsf{stepChilds}\ cs\ \sigma$, and by Definition 9

$$\mathsf{runSeq}\ (\mathtt{par}\ cs_3\ k)\ \sigma_3 = \mathsf{runSeq}\ (k\, vs_3)\ \tau_3, \qquad (vs_3, \tau_3) = \mathsf{runChilds}\ cs_3\ \sigma_3.$$

So it suffices to show $\mathsf{runChilds}\ cs_3\ \sigma_3 = \mathsf{runChilds}\ cs\ \sigma$ — and that is exactly the children-level statement of this very lemma, whose hypotheses are (W1) and (W2). Apply the children's induction hypothesis; then $vs_3 = vs_0$ and $\tau_3 = \tau_0$, and both conclusions follow.

So the `par` case reduces entirely to the children's case.

Children, empty spine. $\mathsf{stepChilds}\ [\,]\ \sigma = [\,]$. Nothing to prove.

Children, the 2-to-N step. Here is all the content. By Definition 26, a step of $p :: ps$ is *either a step of the head p or a step of the tail ps*, and the two cases are not symmetric. Unfolding what must be proved, by Definition 9,

$$\mathsf{runChilds}\ (p :: ps)\ \sigma = ((a, ws), \tau_2), \qquad (a, \tau_1) = \mathsf{runSeq}\ p\ \sigma, \quad (ws, \tau_2) = \mathsf{runChilds}\ ps\ \tau_1.$$

Sub-case A: the head moved. Say $\mathsf{step}\ p\ \sigma$ contains $\langle p', \sigma' \rangle$, so the new spine is $p' :: ps$ at σ', and we must show $\mathsf{runChilds}\ (p' :: ps)\ \sigma' = \mathsf{runChilds}\ (p :: ps)\ \sigma$. Unfold both:

$$\begin{aligned} \text{LHS} &= ((a', ws'), \tau_2') & (a', \tau_1') &= \mathsf{runSeq}\ p'\ \sigma', \quad (ws', \tau_2') = \mathsf{runChilds}\ ps\ \tau_1' \\ \text{RHS} &= ((a,\ ws\ ), \tau_2\ ) & (a,\ \tau_1\ ) &= \mathsf{runSeq}\ p\ \sigma, \quad\ (ws,\ \tau_2\ ) = \mathsf{runChilds}\ ps\ \tau_1 \end{aligned}$$

From (W2) we have $\mathsf{WellFormed}\ p\ \sigma$, so the head's induction hypothesis — the program-level statement applied to p — gives $\mathsf{runSeq}\ p'\ \sigma' = \mathsf{runSeq}\ p\ \sigma$, that is $a' = a$ and $\tau_1' = \tau_1$. But then the tail starts from the *same* memory in both runs, so $ws' = ws$ and $\tau_2' = \tau_2$, and the two sides are identical.

No independence was used. The head is the child runSeq runs first anyway, so stepping it is exactly what runSeq was going to do.

Sub-case B: the tail moved. This is the hard case, and it is precisely Remark 16. Say $\mathsf{stepChilds}\ ps\ \sigma$ contains (ps_3, σ_3), so the new spine is $p :: ps_3$ at σ_3, and we must show

$$\mathsf{runChilds}\ (p :: ps_3)\ \sigma_3 = \mathsf{runChilds}\ (p :: ps)\ \sigma.$$

Stop and look at the difficulty. The tail took its step **at** σ — that is where the machine is. But $\mathsf{childFootprints}$ and $\mathsf{runChilds}$ measure the tail **at** $\tau_1 = (\mathsf{runSeq}\ p\ \sigma).2$, after the head has finished. Two different memories: (W1) is stated about the second, and the step happened in the first.

Before the general argument, the same thing on numbers.

> *Concrete instance.* Take $p = \mathtt{write}\ 5\ 1\ \cdots$ and $ps = \Pi[\,q\,]$ with $q = \mathtt{read}\ 7\ (\lambda v.\ \mathtt{write}\ 8\ v\ \cdots)$, at $\sigma = \sigma_0$.
>
> The tail steps: q performs its read of cell 7 *at* σ_0, getting $\sigma_0\, 7$. A read does not change memory, so $\sigma_3 = \sigma_0$ and $q_3 = \mathtt{write}\ 8\ (\sigma_0\, 7)\ \cdots$.
>
> *Left-hand side*, $\mathsf{runChilds}\ \Pi[\,p\ q_3\,]\ \sigma_0$: run p from σ_0, reaching $\sigma_1 = \sigma_0[5 \mapsto 1]$; then run q_3 from σ_1, which writes $\sigma_0\, 7$ into cell 8. Final memory $\sigma_1[8 \mapsto \sigma_0 7]$.
>
> *Right-hand side*, $\mathsf{runChilds}\ \Pi[\,p\ q\,]\ \sigma_0$: run p, reaching σ_1; then run q from σ_1, which reads cell 7 — getting $\sigma_1\, 7$ — and writes that into cell 8. Final memory $\sigma_1[8 \mapsto \sigma_1 7]$.
>
> These agree **if and only if** $\sigma_1\, 7 = \sigma_0\, 7$, and that holds because $7 \notin W(\mathsf{accesses}\ p\ \sigma_0) = \{5\}$: clause (C2) of Definition 17 together with Lemma 32.
>
> That single equation $\sigma_1\, 7 = \sigma_0\, 7$ is the entire difficulty of the theorem, in one line. Everything that follows is machinery for saying it in general.

The general argument, in three moves.

Move 1: carry the tail's data back to where it actually ran. From (W1), the head's footprint $\mathsf{accesses}\ p\ \sigma$ is independent of every later child's footprint; combining those over the union — footprints combine by concatenation, so a union of independent things is independent — gives $\mathsf{Indep}\ (\mathsf{accesses}\ p\ \sigma)\ (\mathsf{accChilds}\ ps\ \tau_1)$. Now Lemma 35, at the children level, gives

$$\mathsf{accChilds}\ ps\ \tau_1 = \mathsf{accChilds}\ ps\ \sigma \quad \text{and} \quad (\mathsf{runChilds}\ ps\ \tau_1).1 = (\mathsf{runChilds}\ ps\ \sigma).1,$$

so the tail's footprint and answer are the same whether measured at τ_1 or at σ. By Lemma 41 the tail's *hypotheses* — its own well-formedness and its own $\mathsf{IndepAll}$ — transport the same way. The tail's data now lives at the memory where its step actually happened.

Move 2: the head does not notice. The tail's step writes only inside the tail's own footprint — that is conclusion (6) applied one level down, together with Lemma 32 for the single step. Since Indep separates the head's footprint from the tail's, Lemma 34 taken with $F :=$ the tail's writes gives that the head computes the same answer at σ_3 as at σ, and lands on a memory agreeing with τ_1 outside the tail's footprint.

Move 3: replay the tail's step where it is measured. The tail's step, legal at σ, is also available at τ_1, because by Move 1 the tail behaves identically at both. Applying the tail's induction hypothesis at τ_1 gives $\mathsf{runChilds}\ ps_3$ (the corresponding memory) $= \mathsf{runChilds}\ ps\ \tau_1$. To know that corresponding memory is *exactly* the right one, and not merely one agreeing outside some set, takes conclusion (6) of this very lemma. □

5.4.1 Why the two conclusions are proved together

Remark 58. Conclusion (6) cannot be dropped and recovered later. Proving only (5) leaves no way to re-establish the hypothesis for the following step, since the program has changed and its footprint with it. And Move 3 above needs the footprint fact at the *same* inductive depth at which it is proving the value fact. Each half is needed to establish the other, so they are carried simultaneously — the same phenomenon as the three conclusions of Lemma 34.

The moral. There is exactly one case in the entire theorem with real content: a child that is *not* the leftmost takes a step, so the machine's memory and runSeq's reference memory disagree. Independence is what makes that disagreement invisible. Every other case is runSeq's own defining equation read in the other direction.

5.5 Piece 4: chaining

One step is not enough; the invariant must survive the whole run. Re-applying Lemma 57 needs $\mathsf{WellFormed}$ at the new configuration, and that needs Bernstein's three conditions *individually* — the single $\mathsf{FDisjoint}$ of (6) bundles reads and writes together and is too coarse to rebuild them.

Lemma 59 (Footprints shrink)**.** *Under the hypotheses of Lemma 57,* $R(\mathsf{accesses}\ P_3\ \sigma_3) \subseteq R(\mathsf{accesses}\ P\ \sigma)$ *and* $W(\mathsf{accesses}\ P_3\ \sigma_3) \subseteq W(\mathsf{accesses}\ P\ \sigma)$.

Proof. The same induction as Lemma 57, tracking reads and writes as two separate containments rather than as one $\mathsf{FDisjoint}$. No case differs in substance; the bookkeeping differs because containment, unlike disjointness from a given F, is what Definition 17 is built from. □

Lemma 60 (Preservation)**.** *If* $\mathsf{WellFormed}\ P\ \sigma$ *and* $\langle P_3, \sigma_3\rangle \in \mathsf{stepC}\ \langle P, \sigma\rangle$ *then* $\mathsf{WellFormed}\ P_3\ \sigma_3$*; and at the children level,* $\mathsf{wfChilds}$ *together with* $\mathsf{IndepAll}$ *of the child footprints is preserved.*

Proof. Mutual induction on P.

For `read` $l\ k$, Definition 21 *defines* $\mathsf{WellFormed}\ (\texttt{read}\ l\ k)\ \sigma$ to be $\mathsf{WellFormed}\ (k\,(\sigma\,l))\ \sigma$, which is the successor's obligation; nothing to do. Likewise `write`.

For par at a join, the successor is $k\,vs$ at σ, and case (ii) of Lemma 57 showed runChilds $cs\ \sigma = (vs, \sigma)$; so conjunct (W3) *is* WellFormed $(k\,vs)\ \sigma$.

For par with a child moved, the successor is par $cs_3\ k$ at σ_3 and we need its three conjuncts. The third is WellFormed $(k\,vs_3)\ \tau_3$, and Lemma 57 gave runChilds $cs_3\ \sigma_3 =$ runChilds $cs\ \sigma$, so it is (W3) unchanged. The first two come from the children's induction hypothesis. The one genuinely new fact needed is that IndepAll survives — and by Lemma 59 each child's footprint only shrinks, while by Lemma 45 independence is monotone under shrinking either side. So the obligation can only get easier. □

Lemma 61 (Chaining)**.** *If* WellFormed $P\ \sigma$ *and* Reaches $\langle P, \sigma\rangle\ \langle P', \sigma'\rangle$ *then* WellFormed $P'\ \sigma'$ *and* runSeq $P'\ \sigma' =$ runSeq $P\ \sigma$*; moreover* accesses $P'\ \sigma'$ *is contained in* accesses $P\ \sigma$ *componentwise.*

Proof. Induction on the derivation of Reaches, which by Definition 29 has two rules.

Rule refl. Then $P' = P$ and $\sigma' = \sigma$; all three claims are the hypothesis or trivial equalities.

Rule tail. We have Reaches $\langle P, \sigma\rangle\ c'$ and the final configuration is some $c'' \in$ stepC c'. By the induction hypothesis $c' = \langle P'', \sigma''\rangle$ with WellFormed $P''\ \sigma''$ and runSeq $P''\ \sigma'' =$ runSeq $P\ \sigma$ and the containment. Now Lemma 60 applied to the last step gives WellFormed at c'', Lemma 57(5) gives runSeq unchanged across it, and Lemma 59 composes the containment. The **wrong** case is vacuous, since stepC **wrong** $= [\,]$. □

The moral. Lemma 61 resolves what looks like a circularity in the design. WellFormed was defined by measuring footprints along runSeq, a *solo*, left-first run; the theorem's conclusion is about *interleaved* runs under Reaches, where no such order exists. There is no obvious reason a hypothesis stated about the first should still hold at a configuration reached by the second. Lemma 61 says it does: WellFormed holds at every configuration of every interleaving, and runSeq reads the same value at each of them (Figure 3). So the hypothesis is available wherever the proof needs it, and the quantity the theorem is about never moves.

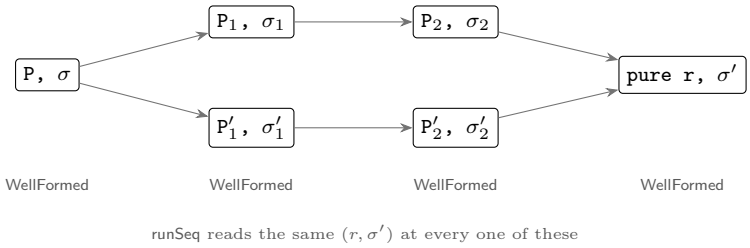


Figure 3: Lemma 60 carries WellFormed across one edge and Lemma 57 carries runSeq's value; Lemma 61 iterates both along any path. Different paths may visit different configurations, but every path ends at the one outcome runSeq named at the start.

5.6 Safety: the forward direction

Proof of Theorem 52, direction $\Longrightarrow$. Suppose WellFormed $p\ \sigma$ and Reaches $\langle p, \sigma\rangle\ \langle$pure $r, \sigma'\rangle$. Apply Lemma 61 to that run, with $P' :=$ pure r and the memory σ'. It gives

$$\text{runSeq}\ (\texttt{pure}\ r)\ \sigma' = \text{runSeq}\ p\ \sigma.$$

By Definition 9's first line, runSeq (pure r) $\sigma' = (r, \sigma')$. Substituting, $(r, \sigma') =$ runSeq $p\ \sigma$, as required. □

Three lines. That is the payoff of finding the right invariant: once the conserved quantity is known, safety is immediate. Note that this direction uses *no induction on the program at all* — the induction happened inside Lemmas 57–61.

5.7 Liveness: the backward direction

Scratch work. We are asked to show something is *reachable*, which calls for a construction: exhibit a schedule. And we know which one to try — the left-to-right one, because that is the one runSeq describes. So the content of this direction is only: *the sequential schedule is a legal schedule.* The one thing that could go wrong is tripping the race test, and Lemma 39 rules that out.

Proof of Theorem 52, direction $\Longleftarrow$. By mutual induction on p, with the children-level motive $motive_2$ of Theorem 54.

Case `pure` a. Here runSeq (`pure` a) $\sigma = (a, \sigma)$, so $r = a$ and $\sigma' = \sigma$, and the required run is Reaches $\langle$`pure` $a, \sigma\rangle$ $\langle$`pure` $a, \sigma\rangle$, which is the refl rule.

Case `read` l k. Since runSeq (`read` l k) σ = runSeq $(k\,(\sigma\,l))$ σ, we have $(r, \sigma') =$ runSeq $(k\,(\sigma\,l))$ σ; and since WellFormed (`read` l k) σ = WellFormed $(k\,(\sigma\,l))$ σ, the induction hypothesis applies to $k\,(\sigma\,l)$ at σ and gives Reaches $\langle k\,(\sigma\,l), \sigma\rangle$ $\langle$`pure` $r, \sigma'\rangle$. Prepend one step: $\langle k\,(\sigma\,l), \sigma\rangle \in$ step (`read` l k) σ by Definition 26, so the tail rule and transitivity (Proposition 30) give the run from $\langle$`read` l $k, \sigma\rangle$.

Case `write` l v k. Identical, at the memory $\sigma[l \mapsto v]$.

Case `par` cs k. Write $(vs, \tau) =$ runChilds cs σ. Build the schedule in five steps.

1. *Run all the children to completion.* By $motive_2$, with hypotheses (W1) and (W2) from WellFormed, the children can collectively reach the configuration in which all are finished leaves holding vs, at memory τ.
2. *Check nothing races along the way.* By Lemma 60 at the children level, IndepAll holds at every configuration of that run; by Lemma 39 it makes racingAll false at each one. So no configuration on this path goes to **wrong**.
3. *Lift it to the fork.* Lemma 56(3) now applies, and gives

$$\text{Reaches } \langle \texttt{par}\ cs\ k,\ \sigma\rangle\ \langle \texttt{par}\ (\text{pureChilds}\ vs)\ k,\ \tau\rangle.$$

4. *Fire the join.* At that configuration every child is a `pure` leaf, so every nextAccs is empty and racingAll is false; and allPure is some vs. So Definition 26's second branch fires, with the single successor $\langle k\,vs, \tau\rangle$. That is one more step.
5. *Finish in the continuation.* Conjunct (W3) is WellFormed $(k\,vs)$ τ, and by Definition 9 runSeq (`par` cs k) σ = runSeq $(k\,vs)$ $\tau = (r, \sigma')$. So the continuation's induction hypothesis gives Reaches $\langle k\,vs, \tau\rangle$ $\langle$`pure` $r, \sigma'\rangle$.

Compose steps 3, 4 and 5 by transitivity.

Children, empty spine. runChilds [ ] $\sigma = ((), \sigma)$ and the required run is reflexivity.

Children, the 2-to-N step. Write $(a, \tau_1) =$ runSeq p σ and $(ws, \tau_2) =$ runChilds ps τ_1.

- *Run the head to completion*, applying the *program-level* statement $motive_1$ to p at σ. This is Theorem 52 one level down, which is exactly why the two statements must be proved mutually. It gives Reaches $\langle p, \sigma\rangle$ $\langle$`pure` $a, \tau_1\rangle$.
- *Lift it*, tail standing still, by Lemma 56(1).
- *Run the tail from there.* The tail's hypotheses hold at τ_1, because that is how Definitions 15 and 21 are stated — each child measured from the memory the previous one left. The tail's induction hypothesis gives a collective run of ps from τ_1 to all-finished at τ_2.
- *Lift it*, head standing still, by Lemma 56(2).

Compose by transitivity. The result is $\texttt{pure}\ a$ followed by the finished tail at τ_2, which is $\mathsf{pureChilds}\ (a, ws)$ at τ_2 — and $\mathsf{runChilds}\ (p :: ps)\ \sigma = ((a, ws), \tau_2)$ by Definition 9. □

The moral. The liveness half contains no clever idea. It walks the left-to-right schedule and checks, at each move, that the machine was *allowed* to make it. That is all "the sequential schedule is one of the schedules" means, and it is the half a soundness theorem omits.

5.8 Parallel equals sequential

Theorem 62 (Sequentialization)**.** *If* $\mathsf{WellFormed}\ (\mathsf{parAll}\ ps)\ \sigma$ *then* $\mathsf{parAll}\ ps \parallel\equiv \mathsf{seqAll}\ ps$ *from* σ*. In particular, if* $\mathsf{WellFormed}\ (\mathsf{parPair}\ p\ q)\ \sigma$ *then* $\mathsf{parPair}\ p\ q \parallel\equiv \mathsf{seqPair}\ p\ q$ *from* σ*.*

Scratch work. Theorem 52 says each side has *exactly one* possible outcome, named by runSeq. So to show the two sides have the same outcomes we need only show they have the same runSeq value — and that is pure unfolding, needing no independence at all.

Proof. We prove two claims and then apply Theorem 52 to each side. Take the binary case; the N-ary one is identical with a list in place of a pair. Write $(a, \tau_1) = \mathsf{runSeq}\ p\ \sigma$ and $(b, \tau_2) = \mathsf{runSeq}\ q\ \tau_1$.

Claim 63. $\mathsf{runSeq}\ (\mathsf{parPair}\ p\ q)\ \sigma = \mathsf{runSeq}\ (\mathsf{seqPair}\ p\ q)\ \sigma$.

The parallel side. By Definition 7, $\mathsf{parPair}\ p\ q = \texttt{par}\ \Pi[p\ q]\ (\lambda r.\ \texttt{pure}\ (r.1, r.2.1))$. By Definition 9's children clause, $\mathsf{runChilds}\ \Pi[p\ q]\ \sigma = ((a, (b, ())),\ \tau_2)$. So by the par clause,

$$\mathsf{runSeq}\ (\mathsf{parPair}\ p\ q)\ \sigma = \mathsf{runSeq}\ (\texttt{pure}\ (a, b))\ \tau_2 = ((a, b),\ \tau_2).$$

The sequential side. We need one fact about bind:

$$\mathsf{runSeq}\ (\texttt{bind}\ x\ f)\ \sigma = \mathsf{runSeq}\ (f\ v)\ \tau, \qquad (v, \tau) = \mathsf{runSeq}\ x\ \sigma. \tag{8}$$

Proof of (8)*:* induction on x using Proposition 8(i), which suffices because bind leaves a fork's children untouched. For $\texttt{pure}\ v$ we have $\texttt{bind}\ (\texttt{pure}\ v)\ f = f\,v$ and $\mathsf{runSeq}\ (\texttt{pure}\ v)\ \sigma = (v, \sigma)$, so both sides are $\mathsf{runSeq}\ (f\,v)\ \sigma$. For $\texttt{read}\ l\ k$, bind pushes inside, so the left side is $\mathsf{runSeq}\ (\texttt{bind}\ (k\,(\sigma\,l))\ f)\ \sigma$ and the induction hypothesis applies. write and par have the same shape.

Applying (8) twice to $\mathsf{seqPair}\ p\ q = \texttt{bind}\ p\ (\lambda a.\ \texttt{bind}\ q\ (\lambda b.\ \texttt{pure}\ (a, b)))$:

$$\begin{aligned}\mathsf{runSeq}\ (\mathsf{seqPair}\ p\ q)\ \sigma &= \mathsf{runSeq}\ (\texttt{bind}\ q\ (\lambda b.\ \texttt{pure}\ (a, b)))\ \tau_1 \\ &= \mathsf{runSeq}\ (\texttt{pure}\ (a, b))\ \tau_2 = ((a, b),\ \tau_2).\end{aligned}$$

The two sides are equal, proving Claim 63. Note this is an equality of runSeq *values*, not merely of answers — and note that **independence was never mentioned**. It is a definitional identity between the reference run of a fork and the reference run of a sequence, true even for racy programs, because runSeq is left-to-right either way.

Claim 64. $\mathsf{WellFormed}\ (\mathsf{parPair}\ p\ q)\ \sigma$ *implies* $\mathsf{WellFormed}\ (\mathsf{seqPair}\ p\ q)\ \sigma$.

$\mathsf{seqPair}\ p\ q$ contains no par node of its own, so Definition 21 imposes no independence obligation on it. Unfolding WellFormed along bind — an easy induction, companion to (8) —

$$\mathsf{WellFormed}\ (\texttt{bind}\ x\ f)\ \sigma \iff \mathsf{WellFormed}\ x\ \sigma \wedge \mathsf{WellFormed}\ (f\,v)\ \tau, \qquad (v, \tau) = \mathsf{runSeq}\ x\ \sigma,$$

and applying it twice reduces $\mathsf{WellFormed}\ (\mathsf{seqPair}\ p\ q)\ \sigma$ to exactly $\mathsf{WellFormed}\ p\ \sigma$ and $\mathsf{WellFormed}\ q\ \tau_1$. Both are conjunct (W2) of the hypothesis, which by Definition 21 unfolds

to WellFormed $p\ \sigma \wedge$ WellFormed q (runSeq $p\ \sigma$).2 $\wedge$ `True`. The memories match because runChilds and `bind` thread them in the same left-to-right order. This proves Claim 64.

Putting them together. Fix any r and σ'. Then

$$\begin{aligned}
&\mathsf{Reaches}\ \langle \mathsf{parPair}\ p\ q,\ \sigma\rangle\ \langle \texttt{pure}\ r,\ \sigma'\rangle \\
&\quad\longleftrightarrow\ (r,\sigma') = \mathsf{runSeq}\ (\mathsf{parPair}\ p\ q)\ \sigma && \text{(Thm. 52, hypothesis)} \\
&\quad\longleftrightarrow\ (r,\sigma') = \mathsf{runSeq}\ (\mathsf{seqPair}\ p\ q)\ \sigma && \text{(Claim 63)} \\
&\quad\longleftrightarrow\ \mathsf{Reaches}\ \langle \mathsf{seqPair}\ p\ q,\ \sigma\rangle\ \langle \texttt{pure}\ r,\ \sigma'\rangle && \text{(Thm. 52, Claim 64)}
\end{aligned}$$

Since r and σ' were arbitrary, this is Definition 31. □

Remark 65 (Why the N-ary form is not a corollary of the binary one). It is the other way round. parAll *ps* is a *single* `par` node with all of *ps* as children, so Claim 63 holds for it by the same unfolding (with an induction on *ps* unfolding the nested seqPairs), Claim 64 by the same argument, and Theorem 52 was already stated for arbitrary children spines. There is no separate metatheory for k-way forks and no induction over a nest of binary ones: a k-way fork is one node, and the induction over the children list happened once, in Lemma 57's 2-to-N step.

5.9 Strengthenings

Three refinements come free from the shape of the proof, and each is used in practice.

Proposition 66 (Equal canonical runs suffice)**.** *If* WellFormed $p\ \sigma$, WellFormed $q\ \sigma$ *and* runSeq $p\ \sigma =$ runSeq $q\ \sigma$, *then* $p \parallel\equiv q$ *from* σ.

Proof. Theorem 52 on each side, exactly as in the final display above: both reachability statements are equivalent to the same equation. □

This is the general form of Theorem 62: it is not about `par` against `bind` at all, but about any two well-formed programs with the same reference run. Theorem 62 is the instance where Claim 63 supplies the hypothesis.

Proposition 67 (Congruence)**.** *Outcome equivalence is preserved by a prefix and by a continuation: if* $p \parallel\equiv q$ *from every memory reachable at that point, then* `bind` $r\ (\lambda_.\ p) \parallel\equiv$ `bind` $r\ (\lambda_.\ q)$, *and* `bind` $p\ f \parallel\equiv$ `bind` $q\ f$.

Proof. Both reduce to Proposition 66 via (8): a `bind`'s canonical run is determined by the prefix's canonical run and the continuation's, so equal canonical runs on the pieces give equal canonical runs on the composite, and WellFormed composes the same way. □

Congruence is what lets a program with several parallel regions be sequentialised one region at a time, rather than all at once.

6 Cost: the same operations, fewer rounds

Theorem 62 says the parallel program computes the same answer. It says nothing whatever about speed, and a correct parallel program can easily be slower. So there is a second thing to prove, and it has two halves that pull in opposite directions:

- the total number of memory operations does *not* go down — parallelism reorders work, it does not delete it;
- the number of *rounds* does go down — it becomes the maximum over the children instead of the sum.

The second is the interesting one, and the whole difficulty is to *derive* the maximum rather than to write it into a definition. A cost model that simply declared a fork's span to be max of its children's spans would be assuming the result and proving nothing.

6.1 Work

Definition 68 (Work)**.** work $p\ \sigma$ counts the memory operations p performs along its canonical run:

$$\begin{aligned}
\mathsf{work}\ (\mathtt{pure}\ a)\ \sigma &= 0 \\
\mathsf{work}\ (\mathtt{read}\ l\ k)\ \sigma &= 1 + \mathsf{work}\ (k\,(\sigma\, l))\ \sigma \\
\mathsf{work}\ (\mathtt{write}\ l\ v\ k)\ \sigma &= 1 + \mathsf{work}\ (k\,())\ \sigma[l \mapsto v] \\
\mathsf{work}\ (\mathtt{par}\ cs\ k)\ \sigma &= \mathsf{workChilds}\ cs\ \sigma + \mathsf{work}\ (k\, vs)\ \tau, \qquad (vs, \tau) = \mathsf{runChilds}\ cs\ \sigma \\
\mathsf{workChilds}\ [\,]\ \sigma &= 0 \\
\mathsf{workChilds}\ (p :: ps)\ \sigma &= \mathsf{work}\ p\ \sigma + \mathsf{workChilds}\ ps\ \tau_1, \qquad (_, \tau_1) = \mathsf{runSeq}\ p\ \sigma
\end{aligned}$$

Two features are worth noticing. A fork's work is the *sum* over its children, not the maximum: forking does not remove operations. And workChilds is staggered in exactly the way childFootprints is (Definition 15) — each child measured from the memory the previous one left — because work, like accesses, is measured along the canonical run.

Example 69. For $p = \mathtt{write}\ 5\ 1\ (\lambda_.\ \mathtt{pure}\ ())$ we get $\mathsf{work}\ p\ \sigma_0 = 1$, and for $q = \mathtt{read}\ 7\ (\lambda v.\ \mathtt{write}\ 8\ v\ (\lambda_.\ \mathtt{pure}\ ()))$ we get $\mathsf{work}\ q\ \sigma = 2$ at every σ. So $\mathsf{work}\ (\mathsf{parPair}\ p\ q)\ \sigma_0 = (1 + 2) + 0 = 3$: three operations, whether they are run side by side or one after the other.

Theorem 70 (Work is invariant)**.** $\mathsf{work}\ (\mathsf{parAll}\ ps)\ \sigma = \mathsf{work}\ (\mathsf{seqAll}\ ps)\ \sigma$*, and in particular* $\mathsf{work}\ (\mathsf{parPair}\ p\ q)\ \sigma = \mathsf{work}\ (\mathsf{seqPair}\ p\ q)\ \sigma$*.* ***No independence hypothesis is needed.***

Proof. First, work adds across a `bind`:

$$\mathsf{work}\ (\mathtt{bind}\ x\ f)\ \sigma = \mathsf{work}\ x\ \sigma + \mathsf{work}\ (f\, v)\ \tau, \qquad (v, \tau) = \mathsf{runSeq}\ x\ \sigma. \tag{9}$$

This is an induction on x of the same shape as (8): at `pure` v the left side is $\mathsf{work}\ (f\, v)\ \sigma$ and the right side is $0 + \mathsf{work}\ (f\, v)\ \sigma$; at a `read` or `write` both sides gain the same 1 and the induction hypothesis closes the case; at a `par` the children's summand is untouched by `bind` and the continuation's is the induction hypothesis.

Applying (9) along $\mathsf{seqAll}\ ps$ turns the right-hand side into the staggered sum $\sum_i \mathsf{work}\ p_i\ \tau_i$, where τ_i is the memory left by the first i children — which is literally $\mathsf{workChilds}\ ps\ \sigma$ by Definition 68. And $\mathsf{parAll}\ ps$ is a single `par` node with a `pure` continuation, so its work is $\mathsf{workChilds}\ ps\ \sigma + 0$. The two agree.

The one thing to check is that the two sides measure their summands from the *same* memories, and they do, because by Claim 63 the two sides have the same canonical run, and runChilds and `bind` thread memory in the same left-to-right order. □

The moral. Work invariance needs no hypothesis at all, because work is a statement about the canonical run — and the canonical run of a fork *is* the canonical run of the corresponding sequence, racy or not. Everything that needs independence is about the *real* runs.

6.2 Rounds: a second, derived relation

Scratch work. We want to count how long a program takes on a machine with unlimited processors. The semantics of Definition 26 will not do: it advances exactly *one* child per move, and its successor list *branches*, so a path through it is a schedule and counting its length counts a schedule's operations, not elapsed time.

So define a second relation in which one move advances *every* child that is ready, all at once, and which therefore has a *single* successor and does not branch. Iterating a non-branching relation is a count. Then prove the two relations agree — that each of the new relation's big

moves is a legal sequence of the old relation's small moves. Without that proof the round count would be a count of something imaginary.

Definition 71 (Rounds)**.** pstep $p\ \sigma$ returns the residual program and the batch of writes the round performed, or nothing if p is finished:

$$\begin{aligned}
\mathsf{pstep}\ (\mathtt{pure}\ a)\ \sigma &= \mathsf{none} \\
\mathsf{pstep}\ (\mathtt{read}\ l\ k)\ \sigma &= \mathsf{some}\ (k\,(\sigma\, l),\ [\,]) \\
\mathsf{pstep}\ (\mathtt{write}\ l\ v\ k)\ \sigma &= \mathsf{some}\ (k\,(),\ [(l,v)]) \\
\mathsf{pstep}\ (\mathtt{par}\ cs\ k)\ \sigma &= \begin{cases} \mathsf{some}\ (\mathtt{par}\ cs'\ k,\ ws) & \mathsf{pstepChilds}\ cs\ \sigma = \mathsf{some}\ (cs', ws) \\ \mathsf{pstep}\ (k\, vs)\ \sigma & \text{otherwise, the join} \end{cases}
\end{aligned}$$

where pstepChilds advances **every** ready child in one round and concatenates their write batches: a child that can move does, a child that cannot is left where it is, and the round exists as long as at least one child moved.

Definition 72 (Applying a batch)**.** applyAll $\sigma\ ws$ performs a batch of writes on a memory, left to right:

$$\mathsf{applyAll}\ \sigma\ [\,] = \sigma, \qquad \mathsf{applyAll}\ \sigma\ ((l,v) :: ws) = \mathsf{applyAll}\ \sigma[l \mapsto v]\ ws.$$

The left-to-right order inside a batch looks like a choice that could matter, and in general it does: if two children in the same round write the same cell, the last one wins and the answer depends on the order of the spine. Under independence it cannot happen — the children's write sets are disjoint by clause (C1) of Definition 17 — so every address in the batch is distinct and the order is immaterial. This is the first of several places where independence quietly pays for the round model's right to exist.

Remark 73 (Why there are two relations and not one). Figure 4 is the whole distinction. stepChilds returns one successor *per* ready child, so its successor list enumerates schedules; it is what Reaches is built from, and what Theorem 52 quantifies over. pstepChilds returns a *single* successor in which every ready child has moved, so iterating it counts rounds. The word max appears nowhere in Definition 71; it is derived in Theorem 80.

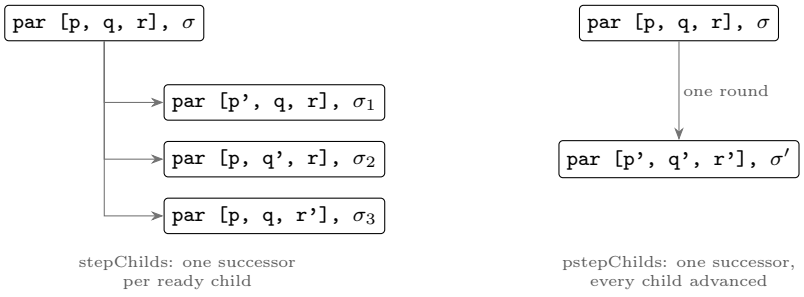


Figure 4: The same fork under the two relations. stepChilds branches, so a path through it is a schedule; pstepChilds does not branch, so iterating it is a count. Lemma 75 is what ties the right-hand arrow back to a path on the left.

Definition 74 (Span)**.** ptime $n\ p\ \sigma$ counts rounds under a fuel bound n: it returns 0 when n is exhausted or when pstep is none, and otherwise 1 plus the count of the residual at fuel $n-1$. The span is

$$\mathsf{time}\ p\ \sigma\ :=\ \mathsf{ptime}\ (\mathsf{work}\ p\ \sigma + 1)\ p\ \sigma.$$

Why fuel, and not a termination argument? Because the recursion is on *rounds*, and a round is not a structurally smaller program — a fork's round rebuilds a fork. Lean needs some decreasing measure, and a counter is the honest one. The fuel never actually binds: a round always performs at least one operation while the program is unfinished (Lemma 78), so work $p\ \sigma$ rounds always suffice and the $+1$ is slack.

6.3 A round is a legal sequence of ordinary steps

Lemma 75 (Realization)**.** *If* WellFormed $p\ \sigma$ *and* pstep $p\ \sigma =$ some (p', ws) *then*

$$\mathsf{Reaches}\ \langle p, \sigma\rangle\ \langle p',\ \mathsf{applyAll}\ \sigma\ ws\rangle;$$

and if pstep $p\ \sigma =$ none *then p is already finished along its canonical run.*

Proof. Mutual induction on p. A leaf's round is a single step, so the `read`, `write` and `pure` cases are immediate, and the join case is one step of Definition 26's second branch. Only the case of a fork with children moving has content: a round in which n children move *at once* has to be exhibited as n consecutive single-child moves, taken in spine order. Four things must hold for that sequence to be a legal run.

(1) *Each move is available.* A child that pstep advances is, by Definition 71, one that step can advance; so it appears in stepChilds's successor list at the moment its turn comes.

(2) *None of them trips the race test.* IndepAll holds at the start by hypothesis, and Lemma 60 carries it across every move in the sequence, so Lemma 39 makes racingAll false at each one. Without this the sequence would reach **wrong** partway through and the run would not exist.

(3) *The memories match up at the end.* Performing the writes one child at a time, in spine order, must leave the same memory as applyAll applying the whole batch at once. The two agree because independence makes the children's write addresses distinct, so neither order can overwrite the other's cell — fact (F1) of Definition 2 then gives equality cell by cell.

(4) *The spine is rebuilt correctly.* This is the 2-to-N step again: the head's moves are lifted to the spine by Lemma 56(1) with the tail standing still, the tail's by Lemma 56(2) with the head standing still, and the two compose by transitivity (Proposition 30).

□

The moral. Lemma 75 is what makes the cost model honest. Without it, time would be the iteration count of a relation invented for the purpose, and a theorem about it would say nothing about the programs. With it, every round is a real run of the real semantics, so "time $p\ \sigma = 3$" means a genuine three-batch execution exists.

Lemma 76 (A round preserves)**.** *Assume the hypotheses of Lemma 75, and write* $\sigma_3 =$ applyAll $\sigma\ ws$*. Then* WellFormed $p'\ \sigma_3$*, the canonical run is unchanged,* runSeq $p'\ \sigma_3 =$ runSeq $p\ \sigma$*, and the residual footprint is contained in the original.*

Proof. A round is a run, by Lemma 75; and Lemma 61 establishes all three statements along an arbitrary run. So there is nothing left to do: the conservation law was already proved for paths, and a round is a path. □

Lemma 77 (Non-interference of a round)**.** *If* WellFormed $p\ \sigma$*, the set F is disjoint from* accesses $p\ \sigma$*, and σ and σ' agree outside F, then* pstep $p\ \sigma =$ pstep $p\ \sigma'$*. Consequently* ptime $n\ p\ \sigma =$ ptime $n\ p\ \sigma'$ *for every fuel n, and* time $p\ \sigma =$ time $p\ \sigma'$*.*

Proof. The statement about one round is an induction mirroring Lemma 34: at a `read` the address is outside F so the two memories give the same value, at a `write` both sides write the same pair, and at a fork the case splits on which children advance — the same ones on both sides, by the children's induction hypothesis. At a join every child is finished, so none has moved memory, and the continuation is entered at the same program and the same memory on both sides.

The statement about the round *count* is an induction on the fuel. After one round the hypotheses must be re-established, and they are: well-formedness by Lemma 76, disjointness from F by the footprint containment in the same lemma, and the agreement of the two memories outside F because applying one batch of writes to two memories that agree outside F leaves them agreeing outside F — every write in the batch is at an address inside the footprint, hence outside F, and is the same write on both sides. □

Lemma 78 (A round makes progress)**.** *Under the hypotheses of Lemma 75,*

$$\mathsf{work}\ p'\ (\mathsf{applyAll}\ \sigma\ ws)\ <\ \mathsf{work}\ p\ \sigma.$$

Consequently $\mathsf{ptime}\ n\ p\ \sigma \le \mathsf{work}\ p\ \sigma$ *for every* n*, the fuel never binds, and* $\mathsf{ptime}\ n\ p\ \sigma = \mathsf{time}\ p\ \sigma$ *for every* $n > \mathsf{work}\ p\ \sigma$*.*

Proof. Mutual induction on p. For a `read` or `write`, work drops by exactly 1 and the claim is immediate. The fork case is the one with content, and the reason is that by Definition 68 a fork's work is a *sum of two summands*,

$$\mathsf{work}\ (\mathtt{par}\ cs\ k)\ \sigma = \underbrace{\mathsf{workChilds}\ cs\ \sigma}_{\text{children}}\ +\ \underbrace{\mathsf{work}\ (k\,vs)\ \tau}_{\text{continuation}},$$

and a round touches only the first. Each summand must be handled separately.

The children's summand strictly decreases. At least one child moved — that is what it means for the round to exist — and by the children's induction hypothesis that child's own work strictly decreased. The children that did *not* move are at the same program, but not at the same memory: the movers' writes have changed the memory underneath them. So their summands are not unchanged for free. They are unchanged by Lemma 77: a non-mover's footprint is disjoint from the movers' by IndepAll, and the new memory agrees with the old outside the movers' footprint, so the non-mover's round count — and with the same argument its work — is the same at both. Hence the children's sum strictly decreases.

The continuation's summand must cancel exactly. Here a bound is not enough. Suppose we only knew that the continuation's work did not grow. Then the total could stay exactly where it was — children's sum down by 1, continuation's up by 1 — and the strict inequality would fail, taking the fuel argument down with it. What is needed is that k is applied to the *same* results and started from the *same* memory before and after the round. And that is exactly what Lemma 76 supplies: equality of canonical runs pins down both components of $\mathsf{runChilds}\ cs\ \sigma$ — the tuple vs passed to k, and the memory τ it starts from. So the continuation's summand is *literally the same number* on both sides, and cancels.

Given strict decrease, the three consequences follow by induction on the fuel: each round spends at least one unit of work, so no more than $\mathsf{work}\ p\ \sigma$ rounds can occur, so any fuel exceeding $\mathsf{work}\ p\ \sigma$ gives the same count. □

6.4 max as a theorem

Definition 79 (The children's span)**.** $\mathsf{ptimes}\ n\ cs\ \sigma$ is the list of the children's own round counts, each measured from the **pre-fork** memory σ, and

$$\mathsf{ptimeChilds}\ n\ cs\ \sigma\ :=\ \mathsf{foldr}\ \max\ 0\ (\mathsf{ptimes}\ n\ cs\ \sigma).$$

Nothing is staggered here, unlike workChilds and childFootprints. That is the point: the children run concurrently, so there is no "memory the previous child left" for a later child to start from. The staggered definitions were about the reference run; this one is about elapsed time. And it is legitimate to measure every child from the same pre-fork memory precisely because, under independence, it makes no difference which memory a child is measured from — Lemma 77.

Theorem 80 (The span law)**.** *For every fuel n, every children spine cs, and every pure continuation,*

$$\mathsf{WellFormed}\ (\mathit{par}\ cs\ (\mathit{pure} \circ f))\ \sigma\ \longrightarrow\ \mathsf{ptime}\ n\ (\mathit{par}\ cs\ (\mathit{pure} \circ f))\ \sigma = \mathsf{ptimeChilds}\ n\ cs\ \sigma.$$

In particular, for two independent branches, $\mathsf{time}\ (\mathsf{parPair}\ p\ q)\ \sigma = \max\ (\mathsf{time}\ p\ \sigma)\ (\mathsf{time}\ q\ \sigma)$.

Proof. Induction on the fuel n.

Base and the finished case. If $\mathsf{pstepChilds}\ cs\ \sigma = \mathsf{none}$ then every child is finished, so every entry of ptimes is 0 and the fold is 0; and the fork takes the join branch into a pure continuation, which has no round either. Both sides are 0. Likewise when the fuel is exhausted.

The step. Otherwise $\mathsf{pstepChilds}\ cs\ \sigma = \mathsf{some}\ (cs', ws)$, and by Definition 71 the fork's round *is* the children's round. So the left-hand side is

$$1 + \mathsf{ptime}\ n\ (\mathtt{par}\ cs'\ (\mathtt{pure} \circ f))\ (\mathsf{applyAll}\ \sigma\ ws),$$

and the induction hypothesis — applicable because the residual fork is well-formed by Lemma 76 — rewrites this to $1 + \mathsf{ptimeChilds}\ n\ cs'\ (\mathsf{applyAll}\ \sigma\ ws)$. What remains to prove is the purely arithmetic statement

$$\mathsf{ptimeChilds}\ (n{+}1)\ cs\ \sigma = 1 + \mathsf{ptimeChilds}\ n\ cs'\ (\mathsf{applyAll}\ \sigma\ ws), \tag{10}$$

and this is an induction over the spine. Take the children one at a time.

- A child that *moved* contributes 1 + (its remaining rounds) on the left, and its remaining rounds on the right.
- A child that did *not* move is finished, contributing 0 on both sides — and its count is unaffected by the movers' writes, by Lemma 77.

So (10) reduces to two facts about max and the successor function:

$$\max\ (1+a)\ (1+b) = 1 + \max\ a\ b, \qquad \max\ (1+a)\ 0 = 1 + \max\ a\ 0.$$

Both are immediate, and together they let the +1 be pulled out of the fold. That is where the maximum comes from: it is not asserted, it is what a fold of max does when every entry either advances by one or is already zero.

Independence is spent exactly once in this proof, in the appeal to Lemma 77, to let each child's count be taken from the pre-fork memory.

The binary form is the two-element case of the general law, since $\mathsf{foldr}\ \max\ 0\ [a, b] = \max\ a\ (\max\ b\ 0) = \max\ a\ b$. There is no separate binary argument. □

Theorem 81 (Sequential composition adds)**.** $\mathsf{time}\ (\mathit{bind}\ p\ f)\ \sigma = \mathsf{time}\ p\ \sigma + \mathsf{time}\ (f\ a)\ \tau_1$ *where* $(a, \tau_1) = \mathsf{runSeq}\ p\ \sigma$*; in particular* $\mathsf{time}\ (\mathsf{seqPair}\ p\ q)\ \sigma = \mathsf{time}\ p\ \sigma + \mathsf{time}\ q\ \tau_1$.

Proof. A round of a bind is a round of the prefix until the prefix finishes, and a round of the continuation thereafter; so the counts add. The induction is on the prefix, transporting the fuel bound with Lemma 78, which says any fuel above the work gives the same count and so the two sides may be compared at a common bound. □

Corollary 82 (Parallel is no slower)**.** *If* WellFormed (parPair p q) σ *then* time (parPair p q) $\sigma \leq$ time (seqPair p q) σ*, with strict inequality when both branches take at least one round. The same holds for* parAll *against* seqAll*.*

Proof. Theorem 80 makes the left-hand side max a b and Theorem 81 makes the right-hand side $a + b$, where $a =$ time p σ and $b =$ time q τ_1 — the two agreeing on where b is measured by Lemma 77. For natural numbers max a $b \leq a+b$ always, and max a $b < a+b$ as soon as both are positive. The N-ary form is an induction on the list whose step is max a (foldr max 0 *rest*) $\leq$ $a + \Sigma$ *rest*. □

The moral. Theorems 70 and 80 together are the separation: the same operations, fewer rounds. Notice which hypotheses each needs. Work invariance needs none, because it is a statement about the canonical run. The span law needs WellFormed, and needs it essentially — as the next example shows, dropping it breaks the conclusion and not merely the proof.

Example 83 (The max law fails for interfering branches)*.* Take p and q both writing the same cell, say $p =$ `write` 5 1 $\cdots$ and $q =$ `write` 5 2 $\cdots$. Definition 71 still assigns each a round count of 1, and still assigns their fork a round count of 1: pstep has no race test, since it was built to count and not to judge.

But in the real semantics the fork does not reach a terminal configuration in one round, or in any number of rounds. By Definition 26 the first thing it does is consult racingAll, which is true — both children's next writes are at cell 5 — so the single successor is **wrong**, which is absorbing.

So dropping WellFormed from Theorem 80 does not merely break the proof; the conclusion becomes a count of rounds of an execution that does not exist. The hypothesis is what makes Lemma 75 available, and Lemma 75 is what ties the count to reality.

7 The whole thing on one page

7.1 Nine lines

1. A memory is a function Loc → Val, and updating one cell leaves the others alone. (Def. 2)
2. A program is a tree: leaves, reads, writes, and forks with N children and a continuation. (Def. 5)
3. runSeq runs that tree left to right. It is a total function, so it names *one* answer. (Def. 9)
4. accesses measures what a program touches along that run; a fork's children are measured staggered. (Defs. 13, 15)
5. Indep is Bernstein: write–write, write–read and read–write disjoint. Shared reads are allowed. (Def. 17)
6. WellFormed asks for Indep at *every* fork, measured along the canonical run. (Def. 21)
7. A step advances one child, or joins, or — if two children's next accesses collide — goes to **wrong**. (Def. 26)
8. runSeq is conserved by every step and WellFormed survives every step, so along every path the one outcome runSeq named at the start is the only one that can be reached, and it is reached. (Thm. 52)
9. A fork and the corresponding sequence have the same runSeq, by unfolding. So they have the same outcomes. (Thm. 62)

Lines 1–7 are definitions; line 8 is the theorem; line 9 is ten lines of unfolding. The one step with real content inside line 8 is sub-case B of Lemma 57: a child that is not the leftmost moves, so the machine's memory and runSeq's reference memory disagree, and independence is what makes the disagreement invisible.

7.2 What rests on what

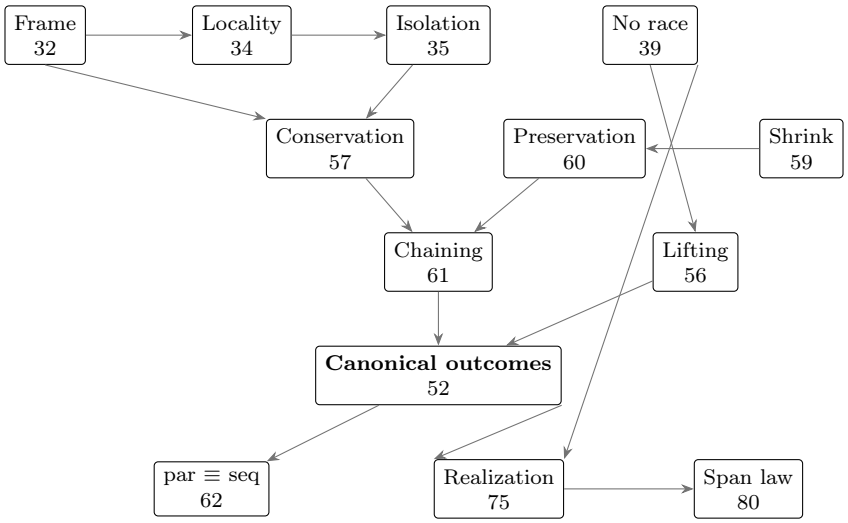


Figure 5: The dependency structure. Frame through Preservation are statements about a *single* step; Chaining, Lifting and everything below them are statements about whole runs. Independence enters at Isolation and No race, and nowhere else.

7.3 Where it breaks

A theorem whose hypothesis is never needed proves nothing. Two counterexamples show that WellFormed is load-bearing, and they fail in two *different* ways — which is why both are needed.

Counterexample 1: the immediate race. Keep $p = \texttt{write}\ 5\ 1\ (\lambda_.\ \texttt{pure}\ ())$ from Example 6 and replace q by

$$q' = \texttt{read}\ 5\ (\lambda v.\ \texttt{write}\ 8\ v\ (\lambda_.\ \texttt{pure}\ ())),$$

which reads cell 5 instead of cell 7. Then $\mathsf{accesses}\ p\ \sigma_0 = (\varnothing, \{5\})$, while $\mathsf{accesses}\ q'\ \tau = (\{5\}, \{8\})$ at *every* memory τ, since q' has no branches. So clause (C2) of Definition 17 fails: $W(p) \cap R(q') = \{5\} \neq \varnothing$. And indeed, at $\texttt{par}\ \Pi[\,p\ q'\,]\ k$ the labels are [write 5] and [read 5], which collide, so racingAll is true and Definition 26 sends the configuration to **wrong** — in *one* step, before anything else can happen. The program has *no* terminating run at all, while $\mathsf{seqPair}\ p\ q'$ has one. So Theorem 52's $\Longleftarrow$ direction fails here: the canonical outcome exists but is not reachable.

The repository's version of this failure is even blunter — two branches that both write cell 0, one storing 1 and one storing 2 — but the point is the same, and p and q' are already in hand.

This is the easy failure, and on its own it would leave a suspicion that WellFormed is merely a convenient way to say "does not crash".

Counterexample 2: the race that does not show up immediately. This is the one that matters, and the repository proves every part of it. Two cells, *shared* = 0 and *own* = 1, and two

programs:

$$\mathit{observer} = \texttt{write}\ \mathit{own}\ 7\ (\lambda_.\ \texttt{read}\ \mathit{shared}\ \texttt{pure})$$
$$\mathit{mutator} = \texttt{write}\ \mathit{shared}\ 99\ \texttt{pure}$$

The observer stamps its own private cell and then *reports* what *shared* holds; the mutator overwrites *shared*. Start from σ_0, in which cell 0 holds 1 and every other cell holds 0, and consider `par` $\Pi[\,\mathit{observer}\ \mathit{mutator}\,]\ k$.

The first step is race-free. The labels are [`write` *own*] and [`write` *shared*] — cells 1 and 0, different cells. So racingAll is **false** and the machine proceeds happily, which is the entire point of this example.

Two of its schedules.

- *Run the mutator to completion first.* Memory becomes cell $0 \mapsto 99$. Then the observer writes cell $1 \mapsto 7$ and reads cell 0, getting 99. The fork joins with the answer $(99, ())$. This schedule **terminates**.
- *Let the observer take its own write first.* Memory becomes cell $1 \mapsto 7$, and the fork is now `read` *shared* against `write` *shared* — the same cell. racingAll is true, and the next step is **wrong**.

The sequential program has one outcome, and it is not that one. Along runSeq the observer runs first and completely, so its read of *shared* happens *before* the mutator's write and sees the starting value 1. Hence runSeq (seqPair *observer mutator*) σ_0 has answer $(1, ())$, and by Theorem 52 — which applies, because a `bind` chain contains no fork and so is well-formed for free — that is the *only* outcome it can reach.

So the parallel program reaches $(99, ())$, the sequential program cannot, and parPair $\parallel \equiv$ seqPair is **false** at σ_0. The hypothesis that fails is, as it must be, IndepAll: the observer reads $\{\mathit{shared}\}$ and the mutator writes $\{\mathit{shared}\}$, so clause (C3) of Definition 17 fails — not at the first step, but one step further in.

The moral. Counterexample 2 is why the race test alone is not the hypothesis. Its first step is *race-free*, so no amount of checking the step in front of you can rule it out. The hypothesis has to look at the whole footprint of each child, along its whole run, which is exactly what Definitions 13 and 15 do and why they are stated the way they are.

The two counterexamples also break Theorem 52 in two *different* places. Counterexample 1 kills the $\Longleftarrow$ direction: the canonical outcome is never reached, because every schedule aborts at once. Counterexample 2 kills the $\Longrightarrow$ direction: a schedule reaches an outcome that is *not* the canonical one. A theorem proving only the first would still hold of Counterexample 1, and would therefore be compatible with a parallel program that simply never runs. That is the sense in which one-sided soundness is not enough, promised back in §1.